

Java Concurrency & GC - Complete Study Guide

The Complete Guide to Java Concurrency & Garbage Collection

A Comprehensive Deep-Dive Study Resource

 Table of Contents

PART 1: JAVA CONCURRENCY - THE COMPLETE GUIDE

PART 2: GARBAGE COLLECTION - THE COMPLETE GUIDE

PART 1: JAVA CONCURRENCY - THE COMPLETE GUIDE

Chapter 1: Threading Fundamentals

1.1 Understanding Threads and Processes

1.2 Thread Lifecycle and States

1.3 Creating Threads - All Methods

1.4 Thread Coordination Mechanisms

1.5 Daemon Threads vs User Threads

Chapter 2: The Synchronization Problem

2.1 Race Conditions - The Root Problem

2.2 Memory Visibility Issues

2.3 The Java Memory Model

2.4 Happens-Before Relationship

2.5 synchronized Keyword Deep Dive

2.6 Intrinsic Locks and Monitors

Chapter 3: Advanced Locking Mechanisms

3.1 Explicit Locks - ReentrantLock

3.2 Fair vs Non-Fair Locks

3.3 ReadWriteLock - Optimizing Read-Heavy Workloads

Chapter 4: The Executor Framework

4.1 Why Thread Pools Matter

4.2 Types of Thread Pools

[4.3 ThreadPoolExecutor Configuration](#)

[4.4 Custom Thread Factories](#)

[4.5 Rejection Policies](#)

Chapter 5: Asynchronous Programming

[5.1 Callable and Future](#)

[5.2 CompletableFuture - The Modern Approach](#)

[5.3 Async Composition Patterns](#)

[5.4 Error Handling in Async Code](#)

4. Async Programming

[Callable & Future](#)

[CompletableFuture \(Java 8+\)](#)

PART 2: GARBAGE COLLECTION

1. Memory Management Basics

[Stack vs Heap](#)

[Object Lifecycle](#)

[Heap Structure \(Generational GC\)](#)

2. Reference Types

[Strong Reference \(Default\)](#)

[Weak Reference](#)

[Soft Reference](#)

[Phantom Reference](#)

3. Memory Leaks in Java

[Yes, Java Can Have Memory Leaks!](#)

[Common Causes](#)

[Detection](#)

4. Garbage Collectors

[Types of GC Events](#)

[GC Algorithms](#)

[Choosing a GC](#)

5. GC Tuning

[Heap Sizing](#)

[GC Logging \(Java 9+\)](#)

[Common Tuning Flags](#)

[Tuning Methodology](#)

Red Flags

6. Monitoring & Tools

Command-Line Tools

GUI Tools

Production Monitoring

Best Practices

Memory Management

GC Tuning

Summary

Concurrency Key Points

GC Key Points

Chapter 6: Atomic Variables and CAS

6.1 Compare-And-Swap (CAS) Operation

6.2 Atomic Classes Family

6.3 ABA Problem and Solutions

6.4 Performance Comparison

Chapter 7: Coordination Utilities (Brief Overview)

7.1 CountdownLatch

7.2 CyclicBarrier

7.3 Semaphore

7.4 BlockingQueue

Chapter 8: Common Pitfalls

8.1 Deadlock

5. Atomic Variables

Lock-Free Thread Safety

6. Coordination Utilities

CountDownLatch

CyclicBarrier

Semaphore

BlockingQueue

7. Common Pitfalls

Deadlock

Visibility Issues



The Complete Guide to Java Concurrency & Garbage Collection

A Comprehensive Deep-Dive Study Resource

Version: 2.0 - Expanded Edition Last Updated: 2026-05-30 Estimated Reading Time: 3-4 hours
Pages: ~50 pages



Table of Contents

PART 1: JAVA CONCURRENCY - THE COMPLETE GUIDE

Chapter 1: Threading Fundamentals (Pages 1-8)

[1.1 Understanding Threads and Processes](#) [1.2 Thread Lifecycle and States](#) [1.3 Creating Threads - All Methods](#) [1.4 Thread Coordination Mechanisms](#) [1.5 Daemon Threads vs User Threads](#)

Chapter 2: The Synchronization Problem (Pages 9-16)

[2.1 Race Conditions - The Root Problem](#) [2.2 Memory Visibility Issues](#) [2.3 The Java Memory Model](#)
[2.4 Happens-Before Relationship](#) [2.5 synchronized Keyword Deep Dive](#) [2.6 Intrinsic Locks and Monitors](#)

Chapter 3: Advanced Locking Mechanisms (Pages 17-24)

[3.1 Explicit Locks - ReentrantLock](#) [3.2 Fair vs Non-Fair Locks](#) [3.3 ReadWriteLock - Optimizing Read-Heavy Workloads](#) [3.4 StampedLock - Optimistic Reads](#) [3.5 Lock-Free Programming Basics](#)

Chapter 4: The Executor Framework (Pages 25-30)

[4.1 Why Thread Pools Matter](#) [4.2 Types of Thread Pools](#) [4.3 ThreadPoolExecutor Configuration](#) [4.4 Custom Thread Factories](#) [4.5 Rejection Policies](#)

Chapter 5: Asynchronous Programming (Pages 31-38)

[5.1 Callable and Future](#) [5.2 CompletableFuture - The Modern Approach](#) [5.3 Async Composition Patterns](#) [5.4 Error Handling in Async Code](#) [5.5 CompletionStage Interface](#)

Chapter 6: Atomic Variables and CAS (Pages 39-42)

[6.1 Compare-And-Swap \(CAS\) Operation](#) [6.2 Atomic Classes Family](#) [6.3 ABA Problem and Solutions](#)
[6.4 Performance Comparison](#)

Chapter 7: Coordination Utilities (Pages 43-50)

[7.1 CountdownLatch - One-Time Events](#) [7.2 CyclicBarrier - Reusable Barriers](#) [7.3 Semaphore - Resource Pools](#) [7.4 Phaser - Flexible Coordination](#) [7.5 Exchanger - Thread Pairs](#)

Chapter 8: Concurrent Collections (Pages 51-56)

[8.1 BlockingQueue Interface](#) [8.2 ConcurrentHashMap Deep Dive](#) [8.3 CopyOnWriteArrayList](#) [8.4 Choosing the Right Collection](#)

Chapter 9: Common Pitfalls and Solutions (Pages 57-64)

[9.1 Deadlock - Detection and Prevention](#) [9.2 Livelock and Starvation](#) [9.3 Thread Leaks](#) [9.4 Visibility Problems](#) [9.5 The Double-Checked Locking Problem](#)

PART 2: GARBAGE COLLECTION - THE COMPLETE GUIDE

Chapter 10: Memory Management Fundamentals (Pages 65-72)

[10.1 JVM Memory Architecture](#) [10.2 Stack Memory Deep Dive](#) [10.3 Heap Memory Structure](#) [10.4 Object Allocation Process](#) [10.5 Escape Analysis](#)

Chapter 11: Garbage Collection Theory (Pages 73-80)

[11.1 Mark-Sweep-Compact Algorithm](#) [11.2 Generational Hypothesis](#) [11.3 Young Generation Collection](#) [11.4 Old Generation Collection](#) [11.5 GC Roots and Reachability](#)

Chapter 12: Reference Types (Pages 81-86)

[12.1 Strong References](#) [12.2 Weak References and WeakHashMap](#) [12.3 Soft References for Caching](#) [12.4 Phantom References and Cleanup](#) [12.5 Reference Queues](#)

Chapter 13: Memory Leaks (Pages 87-92)

[13.1 What is a Memory Leak in Java?](#) [13.2 Common Leak Patterns](#) [13.3 Detection Techniques](#) [13.4 Heap Dump Analysis](#) [13.5 Prevention Strategies](#)

Chapter 14: GC Algorithms (Pages 93-104)

[14.1 Serial Garbage Collector](#) [14.2 Parallel GC \(Throughput Collector\)](#) [14.3 CMS \(Concurrent Mark Sweep\)](#) [14.4 G1 GC \(Garbage First\)](#) [14.5 ZGC \(Z Garbage Collector\)](#) [14.6 Shenandoah GC](#) [14.7 Epsilon GC \(No-Op\)](#)

Chapter 15: GC Tuning (Pages 105-115)

[15.1 Tuning Methodology](#) [15.2 Heap Sizing Guidelines](#) [15.3 GC Logging and Analysis](#) [15.4 JVM Flags Reference](#) [15.5 Performance Metrics](#)

PART 1: JAVA CONCURRENCY - THE COMPLETE GUIDE

Chapter 1: Threading Fundamentals

1.1 Understanding Threads and Processes

What is a Process?

A **process** is an independent, isolated execution environment that contains: - Its own memory space (heap, stack, code, data) - System resources (file handles, network connections) - At least one thread of execution - Process ID assigned by the operating system

Example: When you run your Java application, the JVM creates a new process.

What is a Thread?

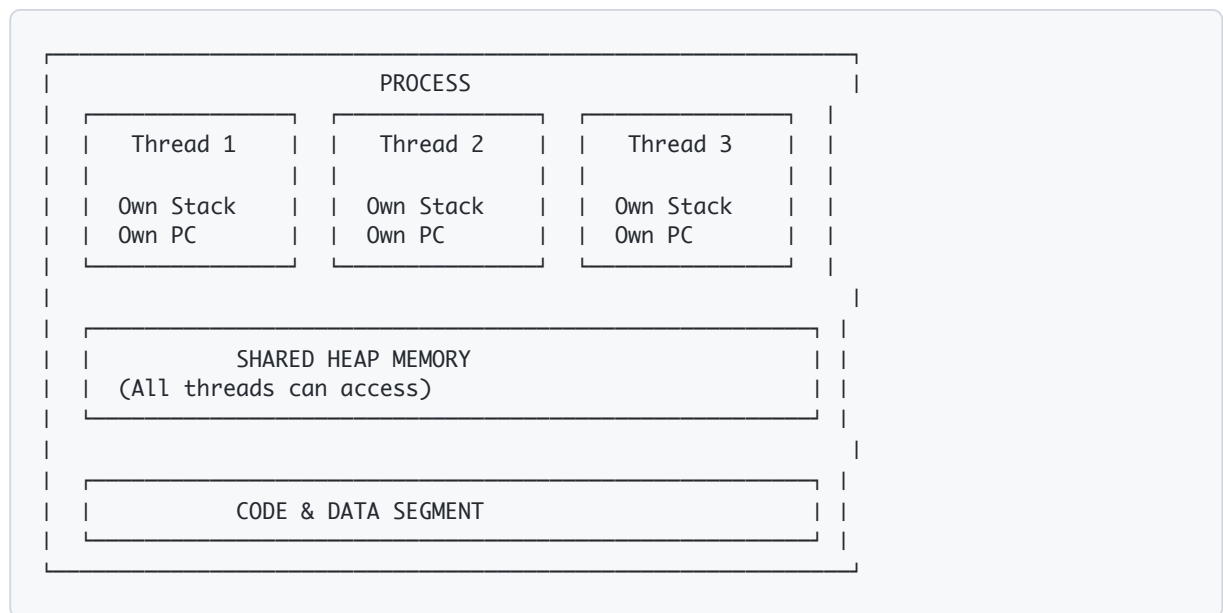
A **thread** is a lightweight unit of execution within a process that: - Shares the process's memory space (heap memory) - Has its own call stack (stack memory) - Has its own program counter (instruction pointer) - Shares resources with other threads in the same process

Analogy: - Process = Restaurant - Threads = Waiters in the restaurant - Shared memory = Kitchen (all waiters access same kitchen) - Stack = Each waiter's notepad

Why Use Threads?

1. **Responsiveness:** Keep UI responsive while doing background work
2. **Resource Sharing:** Threads share memory, making communication easier
3. **Economy:** Creating threads is cheaper than creating processes
4. **Scalability:** Utilize multiple CPU cores effectively
5. **Asynchronous Processing:** Handle multiple operations concurrently

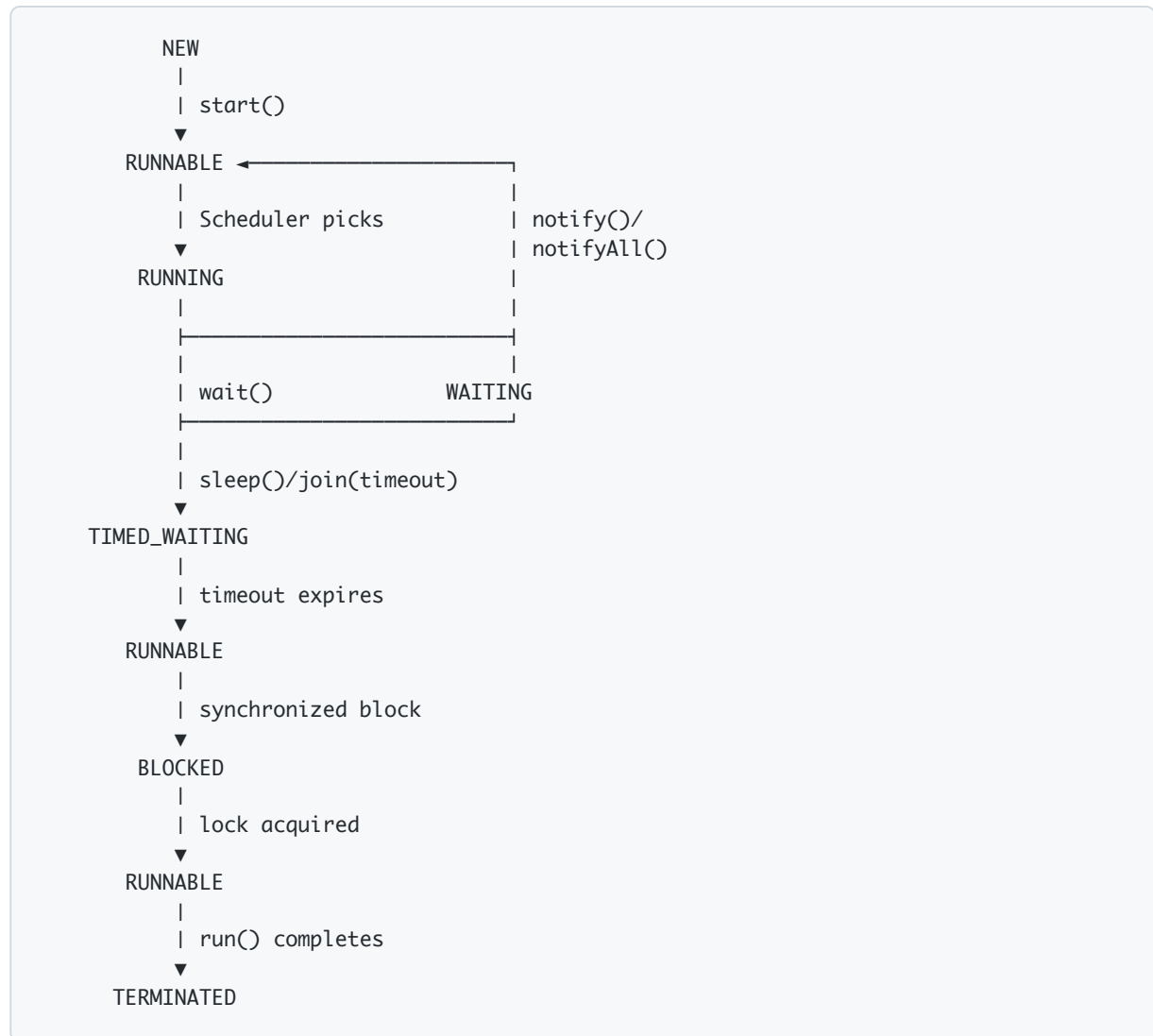
Thread vs Process Comparison



1.2 Thread Lifecycle and States

A thread goes through several states during its lifetime. Understanding these states is crucial for debugging and optimization.

Thread States Diagram



State Descriptions

1. NEW - Thread object created but `start()` not yet called - Not yet scheduled by OS - Example:

```
java Thread t = new Thread() -> System.out.println("Hello"); // State: NEW
```

2. RUNNABLE - `start()` has been called - Thread is ready to run or currently running - OS scheduler decides when to give CPU time - May actually be running or waiting for CPU - Example:

```
java t.start(); // State: RUNNABLE
```

3. BLOCKED - Waiting to acquire a monitor lock - Trying to enter synchronized block/method - Will transition to **RUNNABLE** once lock is acquired - Example: `java synchronized(lock) { // If lock held by another thread // This thread is BLOCKED }`

4. WAITING - Waiting indefinitely for another thread's action - Methods that cause **WAITING**: -

`Object.wait()` without timeout - `Thread.join()` without timeout - `LockSupport.park()` -

Example: `java synchronized(lock) { lock.wait(); // State: WAITING }`

5. TIMED_WAITING - Waiting for specified time period - Methods that cause TIMED_WAITING: -

`Thread.sleep(millis)` - `Object.wait(timeout)` - `Thread.join(timeout)` -
`LockSupport.parkNanos()` - Example: `java Thread.sleep(1000);` // State: TIMED_WAITING for 1 second

6. TERMINATED - Thread has completed execution - `run()` method has exited - Cannot be restarted - Example: `java // After run() completes // State: TERMINATED`

Code Example from ThreadCreationDemo.java

```
// Demonstrating thread states
Thread demoThread = new Thread(() -> {
    try {
        Thread.sleep(2000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "StateDemo");

System.out.println("State after creation: " + demoThread.getState());
// Output: NEW

demoThread.start();
System.out.println("State after start: " + demoThread.getState());
// Output: RUNNABLE

Thread.sleep(100);
System.out.println("State while sleeping: " + demoThread.getState());
// Output: TIMED_WAITING

demoThread.join();
System.out.println("State after completion: " + demoThread.getState());
// Output: TERMINATED
```

Reference: See `ThreadCreationDemo.java` - `demonstrateThreadStates()` method

1.3 Creating Threads - All Methods

There are three primary ways to create threads in Java, each with its own use cases and trade-offs.

Method 1: Extending Thread Class

Approach: Create a subclass of `Thread` and override the `run()` method.

Code Example (from `ThreadCreationDemo.java`):

```

class MyThread extends Thread {
    private String name;

    public MyThread(String name) {
        super(name);
        this.name = name;
    }

    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(name + " executing step " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

// Usage
MyThread thread1 = new MyThread("Thread-1");
thread1.start();

```

Pros: - Simple to understand - Direct access to Thread methods

Cons: - ❌ Cannot extend another class (Java single inheritance) - ❌ Tight coupling to Thread implementation - ❌ Less flexible for task reuse

When to use: Almost never in production code

Method 2: Implementing Runnable Interface

Approach: Implement `Runnable` interface and pass to Thread constructor.

Code Example (from `ThreadCreationDemo.java`):

```

class MyRunnable implements Runnable {
    private String name;

    public MyRunnable(String name) {
        this.name = name;
    }

    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(name + " executing step " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

// Usage
Thread thread2 = new Thread(new MyRunnable("Thread-2"));
thread2.start();

```

Pros: -  Can extend other classes -  Better separation of task from thread -  More flexible and reusable -  Can implement multiple interfaces

Cons: - Slightly more verbose than extending Thread

When to use: Good for most scenarios, especially before Java 8

Method 3: Lambda Expression (Java 8+)

Approach: Use lambda or method reference since `Runnable` is a functional interface.

Code Example (from `ThreadCreationDemo.java`):

```
// Lambda expression
Thread thread3 = new Thread() -> {
    for (int i = 1; i <= 3; i++) {
        System.out.println("Lambda-Thread executing step " + i);
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}, "Lambda-Thread");
thread3.start();

// Even more concise
Thread t = new Thread() -> System.out.println("Quick task");

// Method reference
Thread t2 = new Thread(this::doWork);
```

Pros: -  Most concise and readable -  Modern Java style -  No extra classes needed -  Captures context easily

Cons: - Requires Java 8+ - Can be less clear for complex logic

When to use:  **RECOMMENDED** for new code, especially simple tasks

Method 4: Using Thread Pools (BEST for Production)

Note: We'll cover this extensively in Chapter 4, but here's a preview:

```
ExecutorService executor = Executors.newFixedThreadPool(5);

// Submit task
executor.submit(() -> {
    System.out.println("Task executed by thread pool");
});

executor.shutdown();
```

Why this is best: -  Reuses threads (no overhead of creating new threads) -  Limits resource usage -  Built-in task queue -  Better lifecycle management

When to use:  **ALWAYS** in production code

Comparison Table

Method	Pros	Cons	Use Case
Extend Thread	Simple	No inheritance, tight coupling	Learning only
Implement Runnable	Flexible, reusable	Verbose	Pre-Java 8 code
Lambda	Concise, modern	Requires Java 8+	Simple tasks
Thread Pools	Efficient, scalable	More complex	Production code

Reference: Complete examples in

`src/main/java/org/example/javaconcurrency/concurrency/basics/ThreadCreationDemo.java`

1.4 Thread Coordination Mechanisms

Threads often need to coordinate their execution. Java provides several mechanisms for this.

join() - Wait for Thread Completion

Purpose: Main thread waits for worker thread to finish before proceeding.

Signature:

```
public final void join() throws InterruptedException
public final void join(long millis) throws InterruptedException
public final void join(long millis, int nanos) throws InterruptedException
```

How it works: 1. Calling thread enters WAITING state 2. Waits until target thread enters TERMINATED state 3. Resumes execution after target thread dies

Code Example (from `ThreadCoordinationDemo.java`):

```

Thread task1 = new Thread(() -> {
    System.out.println("Task 1: Starting data download...");
    sleep(1000);
    System.out.println("Task 1: Download completed!");
}, "Downloader");

Thread task2 = new Thread(() -> {
    System.out.println("Task 2: Processing downloaded data...");
    sleep(500);
    System.out.println("Task 2: Processing completed!");
}, "Processor");

// Sequential execution using join()
task1.start();
task1.join(); // Wait for task1 to complete

task2.start();
task2.join(); // Wait for task2 to complete

System.out.println("All tasks completed in sequence!");

```

Output:

```

Task 1: Starting data download...
Task 1: Download completed!
Task 2: Processing downloaded data...
Task 2: Processing completed!
All tasks completed in sequence!

```

Use Cases: - Waiting for worker threads before continuing - Ensuring tasks execute in specific order - Collecting results from multiple threads

Important: Always handle `InterruptedException`

sleep() - Pause Execution

Purpose: Pause current thread for specified time.

Signature:

```

public static void sleep(long millis) throws InterruptedException
public static void sleep(long millis, int nanos) throws InterruptedException

```

Key Points: - Static method - operates on current thread - Thread enters `TIMED_WAITING` state - Does NOT release locks (important!) - Can be interrupted

Code Example:


```
System.out.println("Starting task...");
Thread.sleep(2000); // Sleep for 2 seconds
System.out.println("Task completed after 2 seconds");
```

Common Pattern:

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // Restore interrupt status
    Thread.currentThread().interrupt();
    // Handle interruption
}
```

⚠ Common Mistake:

```
synchronized(lock) {
    Thread.sleep(1000); // KEEPS the lock while sleeping!
    // Other threads CANNOT enter this block
}
```

Better approach:

```
synchronized(lock) {
    lock.wait(1000); // RELEASES lock while waiting
    // Other threads CAN enter this block
}
```

yield() - Hint to Scheduler

Purpose: Suggest to thread scheduler that current thread is willing to give up CPU.

Signature:

```
public static native void yield();
```

Key Points: - Only a **hint** to scheduler (not guaranteed) - Thread stays in **RUNNABLE** state - Rarely needed in practice - Used in spin-wait scenarios

Code Example:

```
while (someCondition) {
    Thread.yield(); // Give other threads a chance
}
```

When to use: - Rarely in modern code - Sometimes in lock-free algorithms - Testing concurrency scenarios

Note: Don't rely on `yield()` for correctness - it's a performance hint only.

1.5 Daemon Threads vs User Threads

Understanding Thread Types

Java has two types of threads:

1. **User Threads** (default)
2. **Daemon Threads**

Key Difference: JVM terminates when all user threads complete, even if daemon threads are still running.

Daemon Threads

Characteristics: - Background service threads - JVM doesn't wait for them to complete - Automatically terminated when JVM exits - Cannot prevent JVM shutdown

Common Use Cases: - Garbage collector thread - Finalizer thread - Background monitoring - Housekeeping tasks - Auto-save features

Code Example (from `ThreadCoordinationDemo.java`):

```
// User thread - JVM waits for it
Thread userThread = new Thread(() -> {
    for (int i = 1; i <= 3; i++) {
        System.out.println("User thread working... " + i);
        sleep(300);
    }
    System.out.println("User thread completed!");
}, "UserThread");

// Daemon thread - JVM doesn't wait
Thread daemonThread = new Thread(() -> {
    for (int i = 1; i <= 10; i++) {
        System.out.println("Daemon thread working... " + i);
        sleep(300);
    }
    System.out.println("Daemon thread completed (may not print!)");
}, "DaemonThread");

daemonThread.setDaemon(true); // MUST be called before start()

System.out.println("Starting both threads...");
userThread.start();
daemonThread.start();

userThread.join();
System.out.println("Main thread exiting (daemon thread will be terminated)");
```

Output:

```
Starting both threads...
User thread working... 1
Daemon thread working... 1
User thread working... 2
Daemon thread working... 2
User thread working... 3
Daemon thread working... 3
User thread completed!
Main thread exiting (daemon thread will be terminated)
Daemon thread working... 4 ← May or may not appear
```

Important Rules

1. Must set before start():

```
Thread t = new Thread(() -> { });
t.setDaemon(true); // OK
t.start();

// WRONG:
Thread t2 = new Thread(() -> { });
t2.start();
t2.setDaemon(true); // IllegalStateException!
```

2. Child threads inherit daemon status:

```
Thread daemon = new Thread(() -> {
    Thread child = new Thread(() -> { });
    // child is automatically a daemon thread
    child.start();
});
daemon.setDaemon(true);
daemon.start();
```

3. Don't rely on daemon threads for critical work:

```
// BAD: Don't do this
Thread saveThread = new Thread(() -> {
    saveImportantData(); // Might be killed mid-save!
});
saveThread.setDaemon(true);
saveThread.start();
```

Checking Thread Type

```
Thread t = Thread.currentThread();
if (t.isDaemon()) {
    System.out.println("I'm a daemon thread");
} else {
    System.out.println("I'm a user thread");
}
```

Reference: Complete examples in

`src/main/java/org/example/javaconcurrency/concurrency/basics/ThreadCoordinationDemo.java`

Chapter 2: The Synchronization Problem

2.1 Race Conditions - The Root Problem

What is a Race Condition?

A **race condition** occurs when: 1. Multiple threads access shared data concurrently 2. At least one thread modifies the data 3. The outcome depends on the timing/ordering of thread execution 4. Results are **non-deterministic** and **incorrect**

Why Do Race Conditions Happen?

The Fundamental Problem: Most operations are **not atomic**.

Consider this simple statement:

```
counter++;
```

This looks like one operation, but it's actually **THREE** at the CPU level:

```
1. READ:   temp = counter    // Read current value
2. MODIFY: temp = temp + 1    // Increment
3. WRITE:  counter = temp     // Write back
```

Interleaving Problem:

```
Initial state: counter = 0

Thread 1          Thread 2
-----          -
READ (0)
INCREMENT (1)
WRITE (1)

READ (0)
INCREMENT (1)
WRITE (1)

Final value: 1 (WRONG! Should be 2)
```

Demonstration from RaceConditionDemo.java

Unsafe Counter (WITHOUT synchronization):

```

class UnsafeCounter {
    private int count = 0;

    // NOT thread-safe!
    public void increment() {
        count++; // This is 3 operations: read-modify-write
    }

    public int getCount() {
        return count;
    }
}

// Test with 10 threads, each incrementing 1000 times
UnsafeCounter unsafeCounter = new UnsafeCounter();

Thread[] threads = new Thread[10];
for (int i = 0; i < 10; i++) {
    threads[i] = new Thread(() -> {
        for (int j = 0; j < 1000; j++) {
            unsafeCounter.increment();
        }
    });
}

// Start all threads
for (Thread thread : threads) {
    thread.start();
}

// Wait for completion
for (Thread thread : threads) {
    thread.join();
}

System.out.println("Expected: 10,000");
System.out.println("Actual: " + unsafeCounter.getCount());
// Output: Actual: 9,847 (or some other number < 10,000)
// DATA LOSS!

```

Actual Output (will vary each run):

```

Expected count: 10,000
Actual count: 9,823
Data lost: 177
❌ THIS IS WRONG! Race condition caused data loss.

```

Why the loss? - Multiple threads read the same value - Both increment it - Both write back - One increment is lost in each collision

Real-World Consequences

1. Banking Application:

```

balance = balance + deposit; // Race condition!
// Two deposits of $100 each might only add $100 total

```

2. Inventory System:

```
inventory--; // Race condition!  
// Two purchases might only decrement once  
// Leads to overselling
```

3. User Registration:

```
if (!users.contains(username)) {  
    users.add(username); // Race condition!  
}  
// Same username might be added twice
```

Reference: Complete demonstration in

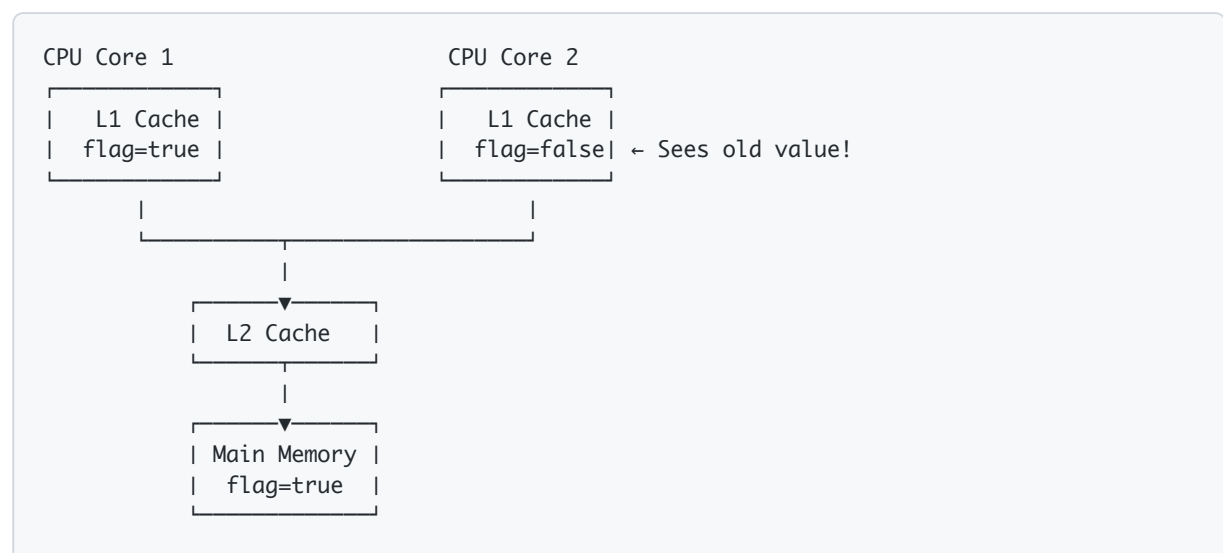
`src/main/java/org/example/javaconcurrency/concurrency/synchronization/RaceConditionDemo.java`

2.2 Memory Visibility Issues

The Visibility Problem

Even if operations were atomic, we still have **visibility issues** due to CPU caching.

Modern CPU Architecture:



Example of Visibility Problem

```
class VisibilityProblem {
    private boolean flag = false; // Shared variable

    // Thread 1 - Writer
    public void writer() {
        // Do some work
        flag = true; // Write to flag
    }

    // Thread 2 - Reader
    public void reader() {
        while (!flag) { // Might loop forever!
            // Waiting...
        }
        System.out.println("Flag is true!");
    }
}
```

What happens? - Thread 1 sets `flag = true` in its CPU cache - Thread 2 keeps reading `flag = false` from its cache - Thread 2 never sees the update! - Infinite loop!

Solution: volatile Keyword

```
class VisibilityFixed {
    private volatile boolean flag = false; // volatile!

    public void writer() {
        flag = true; // Written to main memory
    }

    public void reader() {
        while (!flag) { // Reads from main memory
            // Waiting...
        }
        System.out.println("Flag is true!"); // This WILL execute
    }
}
```

What volatile does: 1. ☒ Reads always from main memory 2. ☒ Writes go directly to main memory 3. ☒ Prevents compiler optimizations that reorder code 4. ☒ **Does NOT make operations atomic**

When to use volatile: - ☒ Simple flags and state variables - ☒ Read-heavy, write-occasional scenarios - ☒ **NOT for compound operations** (counter++, check-then-act)

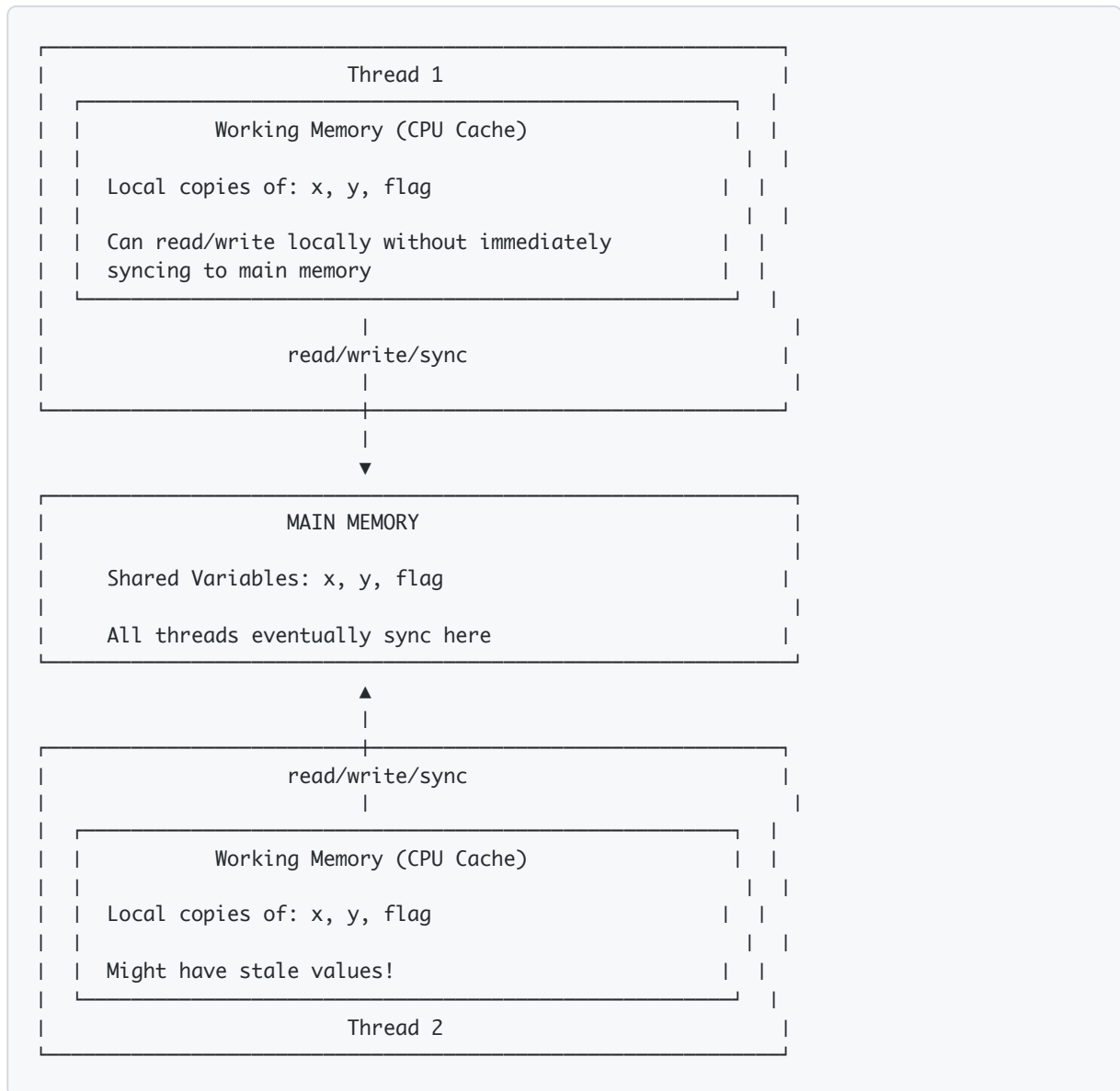
2.3 The Java Memory Model

What is the Java Memory Model (JMM)?

The JMM defines: - How threads interact through memory - When writes become visible to other threads - What ordering guarantees exist - Rules for synchronization

Purpose: Provide consistent behavior across different hardware platforms.

Thread Memory Model Diagram



Memory Operations

1. Read: Transfer from main memory to working memory **2. Load:** Place value in execution engine **3. Use:** Pass value to execution engine **4. Assign:** Receive value from execution engine **5. Store:** Transfer from working memory to main memory **6. Write:** Write to main memory

Problem: These operations can be **reordered** for optimization!

2.4 Happens-Before Relationship

What is Happens-Before?

Happens-before is a guarantee that: - If action A happens-before action B, then - Effects of A are visible to B - A will not be reordered after B

Formal Definition: If one action happens-before another, then the first is visible to and ordered before the second.

Happens-Before Rules

1. Program Order Rule

```
int x = 5;           // A
int y = x + 1;       // B

// A happens-before B (in single thread)
// B sees the write to x
```

2. Monitor Lock Rule

```
synchronized(lock) {
    x = 5;           // A
} // Unlock

synchronized(lock) { // Lock
    y = x + 1;       // B
}

// A happens-before B
// B sees writes from A
```

3. Volatile Variable Rule

```
volatile int v;

v = 5;               // A (write)
int temp = v;        // B (read)

// A happens-before B
// B sees the write
```

4. Thread Start Rule

```
Thread t = new Thread() -> {  
    // B - anything in this thread  
    int y = x; // Sees x=5  
};  
  
x = 5;           // A  
t.start();  
  
// A happens-before B
```

5. Thread Join Rule

```
Thread t = new Thread() -> {  
    x = 5;           // A  
};  
  
t.start();  
t.join();  
int y = x + 1;      // B  
  
// A happens-before B  
// After join(), all writes from t are visible
```

6. Transitivity

```
// If A happens-before B  
// And B happens-before C  
// Then A happens-before C
```

Why This Matters

```
// WITHOUT proper synchronization
class DataRace {
    int x = 0;
    int y = 0;

    // Thread 1
    void writer() {
        x = 1;
        y = 2;
    }

    // Thread 2
    void reader() {
        int a = y; // Might see 2
        int b = x; // Might see 0! (reordering)
        // Possible: a=2, b=0
    }
}

// WITH synchronization
class NoDataRace {
    int x = 0;
    int y = 0;

    synchronized void writer() {
        x = 1;
        y = 2;
    } // Happens-before next lock acquisition

    synchronized void reader() {
        int a = y; // Must see 2
        int b = x; // Must see 1
        // Guaranteed: a=2, b=1
    }
}
```

2.5 synchronized Keyword Deep Dive

How synchronized Works

Purpose: Provides mutual exclusion and memory visibility.

Two Forms:

1. Synchronized Method:

```
public synchronized void method() {  
    // Only one thread at a time  
}  
  
// Equivalent to:  
public void method() {  
    synchronized(this) {  
        // Only one thread at a time  
    }  
}
```

2. Synchronized Block:

```
public void method() {  
    // Code outside block - multiple threads allowed  
  
    synchronized(lockObject) {  
        // Only one thread at a time with this lock  
    }  
  
    // Code outside block - multiple threads allowed  
}
```

The Lock Object

Every object in Java has an **intrinsic lock** (monitor lock).

Lock Rules: 1. Only **one** thread can hold the lock at a time 2. Other threads **block** until lock is released 3. Lock is **reentrant** - same thread can acquire again 4. Lock is released when exiting synchronized block (even via exception)

Example from SynchronizationDemo.java

Method-Level Synchronization:

```

class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    // synchronized method - locks on 'this'
    public synchronized void withdraw(double amount) {
        String threadName = Thread.currentThread().getName();

        System.out.println(threadName + " attempting to withdraw $" + amount);

        if (balance >= amount) {
            System.out.println(threadName + " - Sufficient balance. Processing...");

            // Simulate processing time
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            balance -= amount;
            System.out.println(threadName + " - Withdrawal successful. New balance: $" +
                balance);
        } else {
            System.out.println(threadName + " - Insufficient balance. Current: $" +
                balance);
        }
    }

    public synchronized double getBalance() {
        return balance;
    }
}

// Usage
BankAccount account = new BankAccount(1000);

Thread t1 = new Thread(() -> account.withdraw(600), "ATM-1");
Thread t2 = new Thread(() -> account.withdraw(600), "ATM-2");

t1.start();
t2.start();
t1.join();
t2.join();

System.out.println("Final balance: $" + account.getBalance());

```

Output:

```

ATM-1 attempting to withdraw $600
ATM-1 - Sufficient balance. Processing...
ATM-1 - Withdrawal successful. New balance: $400
ATM-2 attempting to withdraw $600
ATM-2 - Insufficient balance. Current: $400
Final balance: $400
✓ One withdrawal was prevented due to synchronization

```

What happened? 1. ATM-1 acquired lock on account object 2. ATM-2 tried to acquire lock → **BLOCKED** 3. ATM-1 completed withdrawal, released lock 4. ATM-2 acquired lock, saw balance=\$400, rejected 5. Result: **Correct behavior** - no overdraft

Block-Level Synchronization

More Granular Control:

```
class InventorySystem {
    private final Object laptopLock = new Object();
    private final Object mouseLock = new Object();
    private final Object keyboardLock = new Object();

    public void updateProduct(String product, int quantity) {
        // Different locks for different products
        Object lock = getLockForProduct(product);

        synchronized (lock) {
            System.out.println(Thread.currentThread().getName() +
                               " updating " + product + " by " + quantity);

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            System.out.println(Thread.currentThread().getName() +
                               " completed updating " + product);
        }
    }

    private Object getLockForProduct(String product) {
        return switch (product) {
            case "Laptop" -> laptopLock;
            case "Mouse" -> mouseLock;
            case "Keyboard" -> keyboardLock;
            default -> this;
        };
    }
}
```

Benefit: Multiple threads can work on **different products** simultaneously: - Thread-1 updating Laptop (holds laptopLock) - Thread-2 updating Mouse (holds mouseLock) ← Can run in parallel! - Thread-3 updating Laptop (waits for laptopLock)

Reference: Complete code in

```
src/main/java/org/example/javaconcurrency/concurrency/synchronization/SynchronizationDemo.java
```

2.6 Intrinsic Locks and Monitors

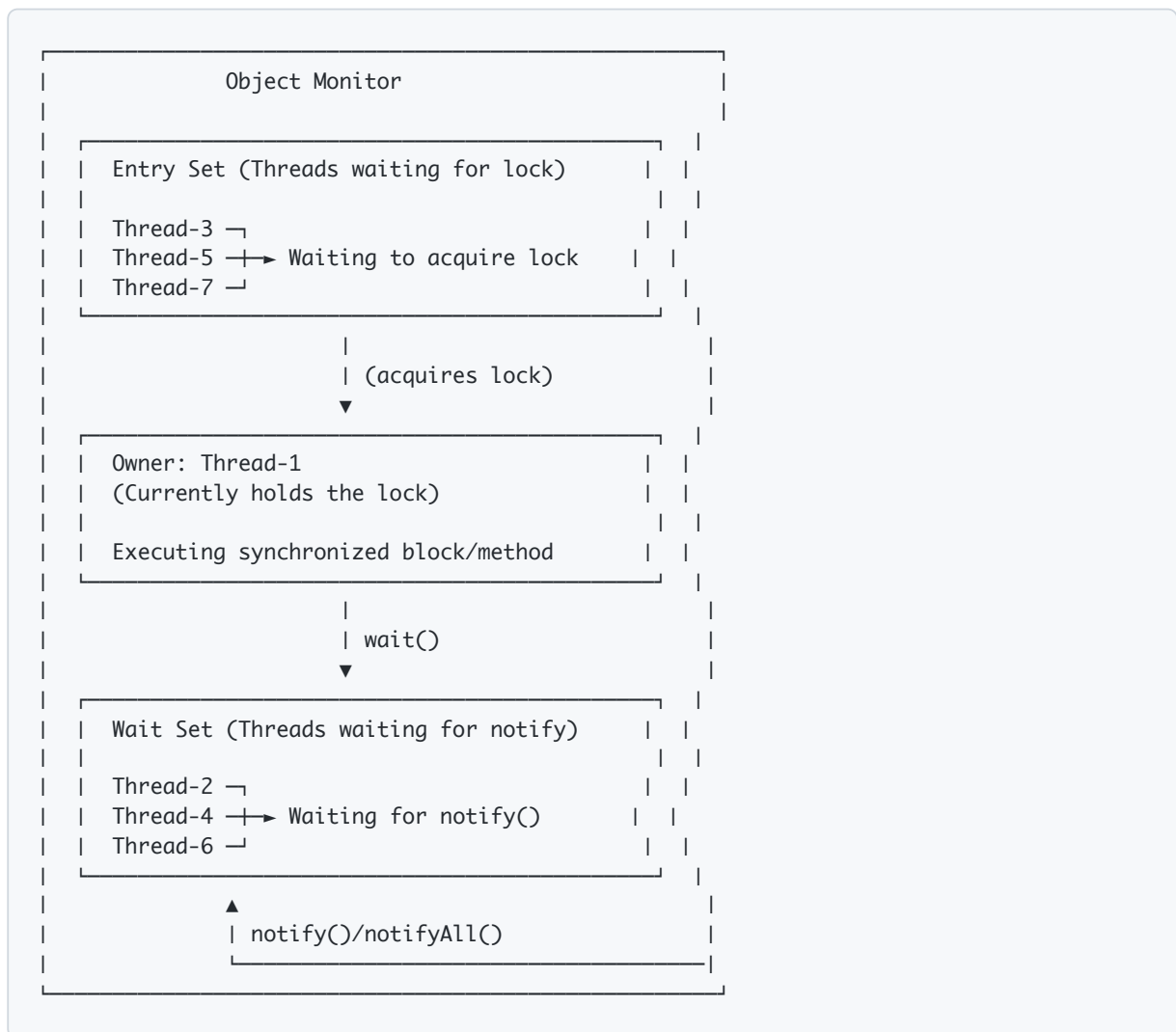
Understanding Monitors

A **monitor** is a synchronization construct that allows threads to:

1. Have mutual exclusion (only one thread in critical section)
2. Wait for a condition to become true
3. Signal other threads when condition changes

Every Java object has an associated monitor.

Monitor Structure



wait(), notify(), and notifyAll()

Purpose: Inter-thread communication within synchronized blocks.

Rules:

1. Must be called within synchronized block/method
2. Must be called on the lock object
3. Calling wait() **releases the lock**
4. Thread enters WAITING state until notified

Signatures:


```
public final void wait() throws InterruptedException
public final void wait(long timeout) throws InterruptedException
public final void notify()
public final void notifyAll()
```

Example - Producer-Consumer:

```
class SharedBuffer {
    private int data;
    private boolean available = false;

    public synchronized void produce(int value) throws InterruptedException {
        // Wait while buffer is full
        while (available) {
            wait(); // Release lock and wait
        }

        data = value;
        available = true;
        System.out.println("Produced: " + value);

        notifyAll(); // Wake up consumers
    }

    public synchronized int consume() throws InterruptedException {
        // Wait while buffer is empty
        while (!available) {
            wait(); // Release lock and wait
        }

        available = false;
        System.out.println("Consumed: " + data);

        notifyAll(); // Wake up producers
        return data;
    }
}
```

Key Points: -  Use `while` (not `if`) for wait condition (guards against spurious wakeups) - 
Use `notifyAll()` (safer than `notify()`) -  Always call within synchronized block on same object

Reentrant Locks

Reentrant means a thread can acquire the same lock multiple times.

```

class ReentrantExample {
    private int count = 0;

    public synchronized void increment() {
        count++;
        // Call another synchronized method
        log(); // OK! Same thread already holds lock
    }

    public synchronized void log() {
        System.out.println("Count: " + count);
        // Can call yet another synchronized method
        validate(); // OK! Reentrant lock
    }

    public synchronized void validate() {
        if (count < 0) throw new IllegalStateException();
    }
}

```

How it works: - Each lock has an **owner thread** and **count** - Same thread acquiring: count++ - Releasing: count-- - Lock fully released when count reaches 0

Static Synchronization

Locks on the Class object:

```

class IdGenerator {
    private static int counter = 0;

    // Locks on IdGenerator.class object
    public static synchronized void generateId(int threadId) {
        System.out.println("Thread-" + threadId + " generating ID...");
        counter++;
        System.out.println("Thread-" + threadId + " generated ID: " + counter);
    }
}

// Equivalent to:
public static void generateId(int threadId) {
    synchronized(IdGenerator.class) {
        System.out.println("Thread-" + threadId + " generating ID...");
        counter++;
        System.out.println("Thread-" + threadId + " generated ID: " + counter);
    }
}

```

Important: Static and instance synchronized methods use **different locks**:

```

class MixedLocks {
    public synchronized void instanceMethod() {
        // Locks on 'this'
    }

    public static synchronized void staticMethod() {
        // Locks on MixedLocks.class
    }
}

// These can run simultaneously!
// Thread-1: instanceMethod() on object1
// Thread-2: staticMethod()
// No contention because different locks!

```

Reference: See

src/main/java/org/example/javaconcurrency/concurrency/synchronization/SynchronizationDemo.java

Chapter 3: Advanced Locking Mechanisms

3.1 Explicit Locks - ReentrantLock

Why ReentrantLock?

Limitations of synchronized: - Cannot interrupt thread waiting for lock - Cannot timeout while waiting for lock - Cannot try to acquire lock without blocking - No fairness guarantee - Lock must be released in same method

ReentrantLock advantages: -  tryLock() - non-blocking attempt -  tryLock(timeout) - timed attempt -  lockInterruptibly() - can be interrupted -  Fair/non-fair policy -  Separate lock/unlock (can unlock in different method) -  Multiple condition variables

Basic Usage

Code Example (from ReentrantLockDemo.java):

```

import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class PrintQueue {
    private final Lock lock = new ReentrantLock();

    public void printDocument(String document) {
        lock.lock(); // Acquire lock
        try {
            System.out.println(Thread.currentThread().getName() +
                               " printing " + document);
            Thread.sleep(500); // Simulate printing
            System.out.println(Thread.currentThread().getName() +
                               " finished printing " + document);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            lock.unlock(); // MUST unlock in finally block!
        }
    }
}

```

Critical Pattern:

```

lock.lock();
try {
    // Critical section
} finally {
    lock.unlock(); // ALWAYS in finally!
}

```

Why finally? Ensures lock is released even if exception occurs.

tryLock() - Non-Blocking Attempt

Purpose: Attempt to acquire lock without blocking.

```

class ResourceManager {
    private final Lock lock = new ReentrantLock();

    public void accessResourceWithTryLock() {
        String threadName = Thread.currentThread().getName();

        if (lock.tryLock()) { // Returns immediately
            try {
                System.out.println(threadName + " acquired lock!");
                Thread.sleep(1000);
                System.out.println(threadName + " releasing lock");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } finally {
                lock.unlock();
            }
        } else {
            System.out.println(threadName + " could NOT acquire lock - doing alternative work");
            // Do something else instead of blocking
        }
    }
}

```

Output (with 2 threads):

```

Worker-1 acquired lock!
Worker-2 could NOT acquire lock - doing alternative work
Worker-1 releasing lock

```

Use Cases: - Avoid deadlocks (don't block waiting) - Responsive UIs (don't freeze) - Alternative fallback logic

tryLock(timeout) - Timed Attempt

Purpose: Wait for lock up to specified time.

```

import java.util.concurrent.TimeUnit;

class DatabaseConnection {
    private final Lock lock = new ReentrantLock();

    public void executeQuery(String query) {
        String threadName = Thread.currentThread().getName();

        try {
            // Try to acquire lock with 2-second timeout
            if (lock.tryLock(2, TimeUnit.SECONDS)) {
                try {
                    System.out.println(threadName + " executing: " + query);
                    Thread.sleep(1500);
                    System.out.println(threadName + " query completed");
                } finally {
                    lock.unlock();
                }
            } else {
                System.out.println(threadName + " timeout - query cancelled");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```

Output (with 2 threads):

```

Query-1 executing: SELECT * FROM users
Query-2 executing: UPDATE users SET ... (after Query-1 completes within 2s)
Query-1 query completed
Query-2 query completed

```

Use Cases: - Database connection pools - Service timeouts - Preventing indefinite waits

lockInterruptibly() - Interruptible Lock

Purpose: Acquire lock but allow interruption while waiting.

```

public void methodWithInterruptibleLock() {
    try {
        lock.lockInterruptibly(); // Can be interrupted while waiting
        try {
            // Critical section
        } finally {
            lock.unlock();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        // Handle interruption
    }
}

```

Use Case: Threads that need to respond to cancellation.

Reference: Complete examples in

```
src/main/java/org/example/javaconcurrency/concurrency/locks/ReentrantLockDemo.java
```

3.2 Fair vs Non-Fair Locks

What is Fairness?

Fair Lock: Threads acquire lock in FIFO order (first-come, first-served) **Non-Fair Lock** (default): No order guarantee (more throughput, less predictable)

Creating Fair Locks

```
// Non-fair (default) - higher throughput
Lock nonFairLock = new ReentrantLock();

// Fair - FIFO order
Lock fairLock = new ReentrantLock(true); // true = fair
```

Demonstration

```
class FairLockExample {
    private final Lock fairLock = new ReentrantLock(true); // Fair

    public void accessResource(int threadNum) {
        fairLock.lock();
        try {
            System.out.println("Thread-" + threadNum + " accessing resource (in order)");
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            fairLock.unlock();
        }
    }
}

// Threads started: Thread-1, Thread-2, Thread-3, Thread-4, Thread-5

// With FAIR lock:
// Thread-1 accessing resource (in order)
// Thread-2 accessing resource (in order)
// Thread-3 accessing resource (in order)
// Thread-4 accessing resource (in order)
// Thread-5 accessing resource (in order)

// With NON-FAIR lock:
// Thread-1 accessing resource (in order)
// Thread-3 accessing resource (in order) ← Out of order!
// Thread-2 accessing resource (in order)
// Thread-5 accessing resource (in order)
// Thread-4 accessing resource (in order)
```

Trade-offs

Fair Lock	Non-Fair Lock
✓ Predictable order	✗ Unpredictable order
✓ No starvation	⚠ Possible starvation
✗ Lower throughput	✓ Higher throughput
✗ More context switches	✓ Fewer context switches

When to use Fair Lock: - Need predictable order - Avoid starvation - Fairness matters more than performance

When to use Non-Fair Lock: - Higher throughput needed (default choice) - Starvation not a concern - Most production scenarios

Reference: See `demonstrateFairLock()` in

`src/main/java/org/example/javaconcurrency/concurrency/locks/ReentrantLockDemo.java`

3.3 ReadWriteLock - Optimizing Read-Heavy Workloads

The Problem with Regular Locks

Scenario: Database cache with many readers, few writers.

With regular lock (`synchronized` or `ReentrantLock`):

```
// INEFFICIENT: Only one reader at a time!
synchronized(lock) {
    return cache.get(key); // Read operation
}
```

Problem: Readers don't conflict with each other! They can all read simultaneously.

ReadWriteLock Solution

Concept: - **Read Lock:** Multiple threads can hold simultaneously (shared) - **Write Lock:** Only one thread, exclusive (blocks all reads and writes)

Interface:

```
public interface ReadWriteLock {
    Lock readLock(); // Shared lock for reading
    Lock writeLock(); // Exclusive lock for writing
}
```


Implementation: ReentrantReadWriteLock

```
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

class CacheWithReadWriteLock {
    private final Map<String, String> cache = new HashMap<>();
    private final ReadWriteLock rwLock = new ReentrantReadWriteLock();

    // Multiple readers can call this simultaneously
    public String get(String key) {
        rwLock.readLock().lock(); // Acquire read lock
        try {
            Thread.sleep(50); // Simulate read time
            return cache.get(key);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return null;
        } finally {
            rwLock.readLock().unlock(); // Release read lock
        }
    }

    // Exclusive write access
    public void put(String key, String value) {
        rwLock.writeLock().lock(); // Acquire write lock
        try {
            System.out.println(Thread.currentThread().getName() +
                               " writing: " + key);
            Thread.sleep(100); // Simulate write time
            cache.put(key, value);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            rwLock.writeLock().unlock(); // Release write lock
        }
    }
}
```

Lock Acquisition Rules

Current State	Read Request	Write Request
No locks	✅ Grant	✅ Grant
Read locks held	✅ Grant	❌ Block (wait)
Write lock held	❌ Block	❌ Block

Visual Representation:

```

Time →

Reader-1: [===== Reading =====]
Reader-2:      [===== Reading =====]      ← Can read simultaneously!
Reader-3:      [===== Reading =====]      ← Can read simultaneously!
Writer-1:      [XXX] ← Waits for all readers
Reader-4:      [=====] ← Waits for writer

```

Performance Comparison

From ReadWriteLockDemo.java:

```
// Test: 8 readers, 2 writers  
  
// With Regular Lock:  
// All operations serialized  
// Time: 4500ms  
  
// With ReadWriteLock:  
// Readers run concurrently!  
// Time: 1200ms  
// Improvement: 73% faster!
```

Complete Performance Test:

```

public class ReadWriteLockDemo {

    private static long testCache(Object cache, String lockType)
        throws InterruptedException {

        Thread[] threads = new Thread[10];
        long startTime = System.currentTimeMillis();

        // 8 readers
        for (int i = 0; i < 8; i++) {
            final int readerId = i + 1;
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 5; j++) {
                    if (cache instanceof CacheWithRegularLock) {
                        ((CacheWithRegularLock) cache).get("user" + (j % 3));
                    } else {
                        ((CacheWithReadWriteLock) cache).get("user" + (j % 3));
                    }
                    try {
                        Thread.sleep(10);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
            }, "Reader-" + readerId);
        }

        // 2 writers
        for (int i = 8; i < 10; i++) {
            final int writerId = i - 7;
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 2; j++) {
                    if (cache instanceof CacheWithRegularLock) {
                        ((CacheWithRegularLock) cache).put("user" + j, "Data-" + j);
                    } else {
                        ((CacheWithReadWriteLock) cache).put("user" + j, "Data-" + j);
                    }
                    try {
                        Thread.sleep(50);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
            }, "Writer-" + writerId);
        }

        for (Thread thread : threads) {
            thread.start();
        }

        for (Thread thread : threads) {
            thread.join();
        }

        return System.currentTimeMillis() - startTime;
    }
}

```

When to Use ReadWriteLock

✅ **Use when:** - Many readers, few writers (read-heavy workload) - Read operations take significant time - Need better concurrency than single lock - Examples: Caches, configuration data, lookup tables

❌ **Don't use when:** - Equal reads and writes - Very short critical sections (overhead not worth it) - Writers are frequent

Performance Characteristics:

Read-to-Write Ratio	Performance Gain
----- -----	
10:1 (read-heavy)	🚀🚀🚀 Excellent
5:1	🚀🚀 Good
1:1 (balanced)	🚀 Marginal
1:5 (write-heavy)	⚠️ May be slower!

Lock Downgrading

Can you downgrade write lock to read lock?

```
rwLock.writeLock().lock();
try {
    // Update cache
    cache.put(key, value);

    // Want to downgrade to read lock
    rwLock.readLock().lock(); // Acquire read lock first
} finally {
    rwLock.writeLock().unlock(); // Release write lock
}

try {
    // Now holding only read lock
    return cache.get(key);
} finally {
    rwLock.readLock().unlock();
}
```

Important: Acquire read lock BEFORE releasing write lock!

Lock Upgrading (NOT Supported!)

```
rwLock.readLock().lock();
try {
    if (needsUpdate) {
        rwLock.writeLock().lock(); // ❌ DEADLOCK!
        // This will hang because you need to release read lock first
    }
} finally {
    rwLock.readLock().unlock();
}
```

Correct approach:

```
rwLock.readLock().lock();
try {
    if (needsUpdate) {
        rwLock.readLock().unlock(); // Release read lock

        rwLock.writeLock().lock(); // Acquire write lock
        try {
            // Do update
        } finally {
            rwLock.writeLock().unlock();
        }

        rwLock.readLock().lock(); // Re-acquire read lock if needed
    }
} finally {
    rwLock.readLock().unlock();
}
```

Reference: Complete implementation in

`src/main/java/org/example/javaconcurrency/concurrency/locks/ReadWriteLockDemo.java`

Chapter 4: The Executor Framework

4.1 Why Thread Pools Matter

The Problem with Manual Thread Creation

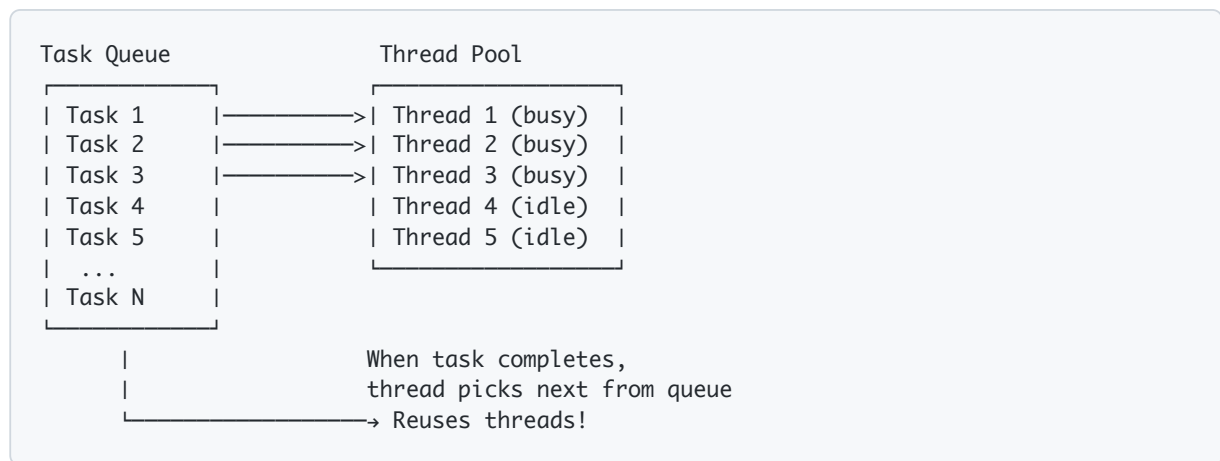
Bad Practice (DON'T DO THIS):

```
for (int i = 0; i < 1000; i++) {
    new Thread(() -> {
        processTask();
    }).start(); // Creates 1000 threads!
}
```

Problems: 1. **Resource Exhaustion:** Each thread consumes memory (~1MB stack) - 1000 threads = ~1GB memory just for stacks! 2. **Thread Creation Overhead:** Creating thread takes time 3. **Context Switching:** Too many threads → excessive context switching 4. **No Lifecycle Management:** Hard to shutdown, cancel, or monitor

The Solution: Thread Pools

Thread Pool Concept:



Benefits: 1. ✅ **Thread Reuse:** No overhead of creating new threads 2. ✅ **Resource Control:** Limit max concurrent threads 3. ✅ **Task Queue:** Tasks wait in queue if all threads busy 4. ✅ **Lifecycle Management:** Easy shutdown, awaitTermination 5. ✅ **Performance:** Better throughput and response time

4.2 Types of Thread Pools

1. FixedThreadPool

Creates a pool with fixed number of threads.

```
ExecutorService executor = Executors.newFixedThreadPool(5);
```

Characteristics: - Fixed number of threads (5 in example) - Unbounded task queue - If all threads busy, tasks wait in queue - Threads are not terminated until shutdown

Code Example (from `ExecutorServiceDemo.java`):

```

ExecutorService executor = Executors.newFixedThreadPool(3);

System.out.println("Submitting 10 tasks to pool of 3 threads...");

for (int i = 1; i <= 10; i++) {
    final int taskId = i;
    executor.submit(() -> {
        System.out.println("Task " + taskId + " started by " +
            Thread.currentThread().getName());

        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println("Task " + taskId + " completed");
    });
}

executor.shutdown(); // No new tasks accepted
executor.awaitTermination(1, TimeUnit.MINUTES);

System.out.println("All tasks completed. Only 3 threads were used.");

```

Output:

```

Submitting 10 tasks to pool of 3 threads...
Task 1 started by pool-1-thread-1
Task 2 started by pool-1-thread-2
Task 3 started by pool-1-thread-3
Task 1 completed
Task 4 started by pool-1-thread-1 ← Thread reused!
Task 2 completed
Task 5 started by pool-1-thread-2
...
All tasks completed. Only 3 threads were used.

```

When to use: - ☒ Known workload with predictable task arrival rate - ☒ Want to limit resource usage - ☒ CPU-bound tasks (use thread count = number of cores)

2. CachedThreadPool

Creates threads as needed, reuses idle threads.

```

ExecutorService executor = Executors.newCachedThreadPool();

```

Characteristics: - No core threads initially - Creates threads on demand - Reuses threads idle for 60 seconds - Terminates idle threads after 60s - Good for many short-lived tasks

Code Example:

```

ExecutorService executor = Executors.newCachedThreadPool();

System.out.println("Submitting 5 quick tasks...");

for (int i = 1; i <= 5; i++) {
    final int taskId = i;
    executor.submit(() -> {
        System.out.println("Quick task " + taskId + " by " +
            Thread.currentThread().getName());
    });
}

Thread.sleep(100);
executor.shutdown();
executor.awaitTermination(10, TimeUnit.SECONDS);

```

When to use: - ☒ Many short-lived tasks - ☒ Unpredictable task arrival rate - ☒ Bursty workloads - ☐  Caution: Can create unlimited threads (OOM risk!)

3. SingleThreadExecutor

Exactly one worker thread, sequential execution.

```

ExecutorService executor = Executors.newSingleThreadExecutor();

```

Characteristics: - Only 1 thread - Tasks execute sequentially (FIFO) - If thread dies, new thread created - Unbounded queue

Code Example:

```

ExecutorService executor = Executors.newSingleThreadExecutor();

System.out.println("Submitting tasks (will execute one at a time)...");

for (int i = 1; i <= 4; i++) {
    final int taskId = i;
    executor.submit(() -> {
        System.out.println("Task " + taskId + " executing...");
        try {
            Thread.sleep(200);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });
}

executor.shutdown();
executor.awaitTermination(10, TimeUnit.SECONDS);

```

Output (sequential execution):


```
Task 1 executing...
Task 2 executing...
Task 3 executing...
Task 4 executing...
```

When to use: -  Need guaranteed sequential execution -  Event-driven systems -  Simplifies concurrency (no race conditions)

4. ScheduledThreadPool

Delayed and periodic task execution.

```
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);
```

Characteristics: - Fixed thread pool - Supports delayed execution - Supports periodic execution

Code Example (from `ExecutorServiceDemo.java`):

```
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

// One-time delayed task
System.out.println("Scheduling task to run after 1 second...");
scheduler.schedule(() -> {
    System.out.println("Delayed task executed!");
}, 1, TimeUnit.SECONDS);

// Periodic task (fixed rate)
System.out.println("Scheduling periodic task (every 500ms)...");
ScheduledFuture<?> periodicTask = scheduler.scheduleAtFixedRate(() -> {
    System.out.println("Periodic task tick - " + System.currentTimeMillis());
}, 0, 500, TimeUnit.MILLISECONDS);

// Let it run for a while
Thread.sleep(2500);

// Cancel periodic task
periodicTask.cancel(false);

scheduler.shutdown();
scheduler.awaitTermination(5, TimeUnit.SECONDS);
```

Methods:

```
// Run once after delay
schedule(Runnable task, long delay, TimeUnit unit)

// Run once after delay, return result
schedule(Callable<V> task, long delay, TimeUnit unit)

// Periodic execution (fixed rate)
// If task takes longer than period, next execution waits
scheduleAtFixedRate(Runnable task, long initialDelay,
                    long period, TimeUnit unit)

// Periodic execution (fixed delay between executions)
scheduleWithFixedDelay(Runnable task, long initialDelay,
                      long delay, TimeUnit unit)
```

Fixed Rate vs Fixed Delay:

Fixed Rate (every 100ms):

Task:	[===]	[===]	[===]	[===]	
Time:	0	100	200	300	400 500
	↑	↑	↑	↑	↑

Start times are fixed

Fixed Delay (100ms between tasks):

Task:	[===]	[===]	[=====]	[=====]	[=====]	[=====]
Time:	0	100	200	300	400	500 600

↑ 100ms delay after completion

When to use: - ☒ Scheduled jobs (cron-like) - ☒ Health checks, monitoring - ☒ Auto-save, cleanup tasks - ☒ Rate-limited operations

Reference: Complete examples in

`src/main/java/org/example/javaconcurrency/concurrency/executors/ExecutorServiceDemo.java`

4.3 ThreadPoolExecutor Configuration

Understanding ThreadPoolExecutor

All `Executors.newXXX()` methods create instances of `ThreadPoolExecutor` :

```
public ThreadPoolExecutor(
    int corePoolSize,           // Minimum threads to keep alive
    int maximumPoolSize,       // Maximum threads allowed
    long keepAliveTime,        // Time idle threads wait before termination
    TimeUnit unit,             // TimeUnit for keepAliveTime
    BlockingQueue<Runnable> workQueue, // Task queue
    ThreadFactory threadFactory, // Factory to create new threads
    RejectedExecutionHandler handler // Policy when queue is full
)
```

Core Parameters

1. **corePoolSize** - Minimum number of threads to keep in pool - Even if idle, these threads are not terminated - Threads created on demand up to this number
2. **maximumPoolSize** - Maximum number of threads allowed - Additional threads created if queue is full - Only relevant if using bounded queue
3. **keepAliveTime** - How long excess threads (beyond core) wait for tasks - After timeout, excess threads are terminated - Doesn't apply to core threads (unless `allowCoreThreadTimeOut(true)`)
4. **workQueue** - Where tasks wait if all threads are busy - Types: - `LinkedBlockingQueue` - Unbounded (default for `FixedThreadPool`) - `ArrayBlockingQueue` - Bounded - `SynchronousQueue` - No capacity (default for `CachedThreadPool`) - `PriorityBlockingQueue` - Priority-ordered

Example: Custom Thread Pool

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(  
    5, // corePoolSize: 5 threads minimum  
    10, // maximumPoolSize: 10 threads maximum  
    60, TimeUnit.SECONDS, // Keep alive: 60 seconds  
    new ArrayBlockingQueue<>(100), // Queue capacity: 100 tasks  
    new ThreadFactory() {  
        private AtomicInteger counter = new AtomicInteger(0);  
  
        @Override  
        public Thread newThread(Runnable r) {  
            Thread t = new Thread(r, "CustomPool-" + counter.incrementAndGet());  
            t.setDaemon(false);  
            t.setPriority(Thread.NORM_PRIORITY);  
            return t;  
        }  
    },  
    new ThreadPoolExecutor.CallerRunsPolicy() // Rejection policy  
);
```

Thread Pool Sizing Guidelines

CPU-Bound Tasks:

Optimal Threads = Number of CPU Cores

```
int cores = Runtime.getRuntime().availableProcessors();  
ExecutorService executor = Executors.newFixedThreadPool(cores);
```

I/O-Bound Tasks:

Optimal Threads = Number of Cores * (1 + Wait Time / Compute Time)

```
// Example: 8 cores, 80% I/O wait
// Optimal = 8 * (1 + 4) = 40 threads
ExecutorService executor = Executors.newFixedThreadPool(40);
```

Mixed Workload: - Start with: `cores * 2` - Monitor and tune based on: - CPU utilization - Task throughput - Response time

4.4 Custom Thread Factories

Why Custom Thread Factory?

Default threads have: - Generic names ("pool-1-thread-1") - Normal priority - Non-daemon status - Default exception handler

Custom factory allows: - Meaningful names (for debugging) - Custom priority - Daemon status - Custom uncaught exception handler

Implementation

```
class CustomThreadFactory implements ThreadFactory {
    private final String namePrefix;
    private final AtomicInteger threadNumber = new AtomicInteger(1);
    private final boolean daemon;

    public CustomThreadFactory(String namePrefix, boolean daemon) {
        this.namePrefix = namePrefix;
        this.daemon = daemon;
    }

    @Override
    public Thread newThread(Runnable r) {
        Thread t = new Thread(r, namePrefix + "-" + threadNumber.getAndIncrement());
        t.setDaemon(daemon);
        t.setPriority(Thread.NORM_PRIORITY);

        // Custom exception handler
        t.setUncaughtExceptionHandler((thread, throwable) -> {
            System.err.println("Exception in " + thread.getName() + ": " + throwable);
            throwable.printStackTrace();
        });

        return t;
    }
}

// Usage
ExecutorService executor = Executors.newFixedThreadPool(5,
    new CustomThreadFactory("MyApp-Worker", false));
```

Benefits: - Better logging (thread names in logs) - Easier debugging (meaningful thread dumps) - Custom error handling - Monitoring integration

4.5 Rejection Policies

When are Tasks Rejected?

Tasks are rejected when: 1. Executor is shut down 2. Queue is full AND all threads are busy (bounded queue only)

Built-in Policies

1. **AbortPolicy** (default) - Throws `RejectedExecutionException` - Caller must handle exception

```
new ThreadPoolExecutor.AbortPolicy()
```

2. **CallerRunsPolicy** - Runs task in caller's thread - Provides throttling (slows down submitter)

```
new ThreadPoolExecutor.CallerRunsPolicy()
```

3. **DiscardPolicy** - Silently discards rejected task - No exception, no execution

```
new ThreadPoolExecutor.DiscardPolicy()
```

4. **DiscardOldestPolicy** - Discards oldest task in queue - Submits new task

```
new ThreadPoolExecutor.DiscardOldestPolicy()
```

Choosing a Policy

Policy	Use When	Pros	Cons
AbortPolicy	Tasks must not be lost	Fail-fast	Caller must handle
CallerRunsPolicy	Can't lose tasks, need throttling	Natural backpressure	Slows caller
DiscardPolicy	Tasks are optional	No exception handling	Silent loss
DiscardOldestPolicy	New tasks more important	Keeps fresh tasks	Old tasks lost

Custom Policy:

```

RejectedExecutionHandler customPolicy = (task, executor) -> {
    // Log rejection
    System.err.println("Task rejected: " + task);

    // Custom logic: save to database, retry later, alert, etc.
    saveForLater(task);
};

ThreadPoolExecutor executor = new ThreadPoolExecutor(
    5, 10, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100),
    Executors.defaultThreadFactory(),
    customPolicy // Custom rejection policy
);

```

Reference: See `ExecutorServiceDemo.java` for complete working examples

Chapter 5: Asynchronous Programming

5.1 Callable and Future

Limitations of Runnable

```

// Runnable - no return value
Runnable task = () -> {
    // Do work
    // Can't return result!
};

```

Problems: - ❌ Cannot return a value - ❌ Cannot throw checked exceptions - ❌ Hard to get results from async tasks

Callable to the Rescue

```

@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception; // Can return value and throw exception
}

```

Example (from `CallableFutureDemo.java`):

```
// Callable returns a value
Callable<Integer> sumTask = () -> {
    System.out.println("Computing sum of 1 to 100...");
    Thread.sleep(1000); // Simulate computation
    int sum = 0;
    for (int i = 1; i <= 100; i++) {
        sum += i;
    }
    return sum; // Returns result!
};

ExecutorService executor = Executors.newSingleThreadExecutor();

Future<Integer> future = executor.submit(sumTask);

System.out.println("Task submitted. Doing other work...");
Thread.sleep(500);

System.out.println("Now waiting for result...");
Integer result = future.get(); // Blocks until result is available

System.out.println("Result: " + result); // Output: Result: 5050
```

Future Interface

```
public interface Future<V> {
    boolean cancel(boolean mayInterruptIfRunning);
    boolean isCancelled();
    boolean isDone();
    V get() throws InterruptedException, ExecutionException;
    V get(long timeout, TimeUnit unit)
        throws InterruptedException, ExecutionException, TimeoutException;
}
```

Methods Explained:

1. get() - Blocking Get

```
Future<String> future = executor.submit(callable);
String result = future.get(); // Blocks until ready
```

2. get(timeout) - Timed Get

```
try {
    String result = future.get(2, TimeUnit.SECONDS);
} catch (TimeoutException e) {
    System.out.println("Task took too long!");
}
```

3. isDone() - Check Completion

```
if (future.isDone()) {  
    String result = future.get(); // Won't block  
}
```

4. cancel() - Cancel Task

```
boolean cancelled = future.cancel(true); // true = interrupt if running  
if (cancelled) {  
    System.out.println("Task was cancelled");  
}
```

invokeAll() - Execute Multiple Tasks

```
List<Callable<String>> tasks = new ArrayList<>();  
  
for (int i = 1; i <= 5; i++) {  
    final int taskId = i;  
    tasks.add(() -> {  
        System.out.println("Task " + taskId + " processing...");  
        Thread.sleep(500 + (taskId * 100));  
        return "Result from Task " + taskId;  
    });  
}  
  
long startTime = System.currentTimeMillis();  
  
// Execute all tasks and wait for all to complete  
List<Future<String>> futures = executor.invokeAll(tasks);  
  
long endTime = System.currentTimeMillis();  
  
System.out.println("All tasks completed in " + (endTime - startTime) + "ms");  
  
// Get results  
for (Future<String> future : futures) {  
    System.out.println(future.get());  
}
```

Output:

```
Task 1 processing...  
Task 2 processing...  
Task 3 processing...  
Task 4 processing...  
Task 5 processing...  
All tasks completed in 1000ms  
Result from Task 1  
Result from Task 2  
Result from Task 3  
Result from Task 4  
Result from Task 5
```


invokeAny() - First to Complete

```
List<Callable<String>> tasks = Arrays.asList(
    () -> { Thread.sleep(1000); return "Task 1"; },
    () -> { Thread.sleep(500); return "Task 2"; },
    () -> { Thread.sleep(800); return "Task 3"; }
);

// Returns result from first completed task
String result = executor.invokeAny(tasks);
System.out.println(result); // "Task 2" (finished first)
```

Reference: Complete examples in

src/main/java/org/example/javaconcurrency/concurrency/async/CallableFutureDemo.java

5.2 CompletableFuture - The Modern Approach

Why CompletableFuture?

Problems with Future: - ❌ Blocking `get()` - can't do non-blocking callbacks - ❌ Can't chain operations - ❌ Can't combine multiple futures - ❌ Limited exception handling

CompletableFuture advantages: - ✅ Non-blocking callbacks - ✅ Chainable operations - ✅ Combine multiple futures - ✅ Rich exception handling - ✅ Completable manually

Basic Usage

Creating CompletableFuture:

```
// Method 1: supplyAsync (returns value)
CompletableFuture<String> future1 = CompletableFuture.supplyAsync(() -> {
    System.out.println("Running in: " + Thread.currentThread().getName());
    sleep(500);
    return "Hello from async task";
});

// Method 2: runAsync (no return value)
CompletableFuture<Void> future2 = CompletableFuture.runAsync(() -> {
    System.out.println("Doing some work...");
});

// Method 3: completedFuture (already completed)
CompletableFuture<String> future3 = CompletableFuture.completedFuture("Already done");
```

Non-Blocking Callbacks:

```

CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
    return "Hello";
});

// thenAccept - non-blocking callback
future.thenAccept(result -> {
    System.out.println("Result: " + result);
});

// Main thread continues without blocking!
System.out.println("Main thread continues...");

```

5.3 Async Composition Patterns

1. thenApply - Transform Result

```

CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> {
        System.out.println("Fetching user ID...");
        sleep(300);
        return 12345;
    })
    .thenApply(userId -> {
        System.out.println("Fetching user details for ID: " + userId);
        sleep(300);
        return "User: John Doe (ID: " + userId + ")";
    })
    .thenApply(userDetails -> {
        System.out.println("Formatting user info...");
        sleep(200);
        return userDetails.toUpperCase();
    });

String result = future.get();
System.out.println("Final result: " + result);

```

Output:

```

Fetching user ID...
Fetching user details for ID: 12345
Formatting user info...
Final result: USER: JOHN DOE (ID: 12345)

```

2. thenCompose - Chain Dependent Futures

Use when: Next step returns a CompletableFuture

```

CompletableFuture<String> orderResult = CompletableFuture
    .supplyAsync(() -> {
        System.out.println("[1] Validating order...");
        sleep(200);
        return "ORDER-12345";
    })
    .thenCompose(orderId -> CompletableFuture.supplyAsync(() -> {
        System.out.println("[2] Processing payment for " + orderId + "...");
        sleep(400);
        return orderId + ":PAID";
    }))
    .thenCompose(paymentInfo -> CompletableFuture.supplyAsync(() -> {
        System.out.println("[3] Updating inventory...");
        sleep(300);
        return paymentInfo + ":INVENTORY_UPDATED";
    }))
    .thenApply(status -> {
        System.out.println("[4] Sending confirmation email...");
        sleep(200);
        return status + ":EMAIL_SENT";
    });

String finalStatus = orderResult.get();
System.out.println("Order status: " + finalStatus);

```

3. thenCombine - Combine Two Independent Futures

```

CompletableFuture<String> weather = CompletableFuture.supplyAsync(() -> {
    System.out.println("Fetching weather data...");
    sleep(500);
    return "Sunny, 25°C";
});

CompletableFuture<String> stocks = CompletableFuture.supplyAsync(() -> {
    System.out.println("Fetching stock price...");
    sleep(700);
    return "AAPL: $150.25";
});

CompletableFuture<String> combined = weather.thenCombine(stocks,
    (weatherData, stockData) -> {
        return "Weather: " + weatherData + ", Stocks: " + stockData;
    });

System.out.println(combined.get());
// Output: Weather: Sunny, 25°C, Stocks: AAPL: $150.25

```

4. allOf - Wait for All

```
CompletableFuture<String> future1 = CompletableFuture.supplyAsync(() -> {
    System.out.println("Fetching weather data...");
    sleep(500);
    return "Sunny, 25°C";
});

CompletableFuture<String> future2 = CompletableFuture.supplyAsync(() -> {
    System.out.println("Fetching stock price...");
    sleep(700);
    return "AAPL: $150.25";
});

CompletableFuture<String> future3 = CompletableFuture.supplyAsync(() -> {
    System.out.println("Fetching news headlines...");
    sleep(400);
    return "Breaking: Tech stocks rally";
});

// Wait for all three to complete
CompletableFuture<Void> allFutures = CompletableFuture.allOf(future1, future2, future3);

allFutures.thenRun(() -> {
    try {
        System.out.println("\nDashboard Data:");
        System.out.println("Weather: " + future1.get());
        System.out.println("Stocks: " + future2.get());
        System.out.println("News: " + future3.get());
    } catch (Exception e) {
        e.printStackTrace();
    }
}).get();
```

5. anyOf - Wait for Any

```
CompletableFuture<String> server1 = CompletableFuture.supplyAsync(() -> {
    sleep(1000);
    return "Response from Server 1";
});

CompletableFuture<String> server2 = CompletableFuture.supplyAsync(() -> {
    sleep(500); // Faster
    return "Response from Server 2";
});

CompletableFuture<String> server3 = CompletableFuture.supplyAsync(() -> {
    sleep(800);
    return "Response from Server 3";
});

// Returns first to complete
CompletableFuture<Object> fastest = CompletableFuture.anyOf(server1, server2, server3);

System.out.println("Fastest response: " + fastest.get());
// Output: Fastest response: Response from Server 2
```

5.4 Error Handling in Async Code

exceptionally - Handle Exceptions

```
CompletableFuture<String> futureWithError = CompletableFuture
    .supplyAsync(() -> {
        System.out.println("Processing payment...");
        sleep(300);
        if (Math.random() > 0.5) {
            throw new RuntimeException("Payment gateway error!");
        }
        return "Payment successful";
    })
    .exceptionally(ex -> {
        System.out.println("Error occurred: " + ex.getMessage());
        return "Payment failed - using backup method";
    })
    .thenApply(result -> {
        System.out.println("Final status: " + result);
        return result;
    });

futureWithError.get();
```

handle - Handle Both Result and Exception

```
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> {
        if (Math.random() > 0.5) {
            throw new RuntimeException("Error!");
        }
        return "Success";
    })
    .handle((result, ex) -> {
        if (ex != null) {
            return "Error handled: " + ex.getMessage();
        } else {
            return "Result: " + result;
        }
    });
```

whenComplete - Side Effect (Success or Failure)

```
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> "Hello")
    .whenComplete((result, ex) -> {
        if (ex != null) {
            System.out.println("Failed with: " + ex);
        } else {
            System.out.println("Succeeded with: " + result);
        }
    });
```

Reference: Complete examples in

`src/main/java/org/example/javaconcurrency/concurrency/async/CompletableFutureDemo.java`

4. Async Programming

Callable & Future

Callable: Like Runnable but returns value and can throw exception

```
Callable<Integer> task = () -> {  
    Thread.sleep(1000);  
    return 42;  
};  
  
Future<Integer> future = executor.submit(task);  
Integer result = future.get(); // Blocks until ready
```

Future Methods: - `get()` - Block until result ready - `get(timeout, unit)` - Block with timeout - `isDone()` - Check if complete - `cancel()` - Attempt to cancel - `isCancelled()` - Check if cancelled

CompletableFuture (Java 8+)

Modern async API with powerful composition:

PART 2: GARBAGE COLLECTION

1. Memory Management Basics

Stack vs Heap

Stack	Heap
Stores local variables, method calls	Stores objects, instance variables
Thread-specific (each thread has own)	Shared across all threads
Fast allocation/deallocation	Slower, managed by GC
Limited size (~1MB default)	Much larger (configurable GB)
LIFO structure	No specific structure
Auto-managed (scope-based)	GC-managed

Example:

```
public void method() {  
    int x = 10;           // Stack  
    String str = "hello"; // str reference on stack, "hello" object on heap  
    Person p = new Person(); // p on stack, Person object on heap  
}  
// x, str, p removed from stack when method returns  
// Objects on heap become eligible for GC when unreachable
```

Object Lifecycle

1. **Created:** `new Object()` - allocated on heap
2. **In Use:** Reachable via references
3. **Unreachable:** No more references (GC eligible)
4. **Finalized:** `finalize()` called (deprecated, don't use)
5. **Collected:** Memory reclaimed by GC

Heap Structure (Generational GC)

```
Heap Memory
├─ Young Generation (short-lived objects)
│  ├─ Eden Space (new objects)
│  ├─ Survivor Space 0 (survived 1 GC)
│  └─ Survivor Space 1 (survived 2+ GCs)
├─ Old Generation (long-lived objects)
│  └─ Tenured Space (survived many GCs)
└─ Permanent Generation (Java 7) / Metaspace (Java 8+)
    └─ Class metadata, static variables
```

Why Generational? - Weak Generational Hypothesis: Most objects die young - Young gen GC (Minor GC): Frequent, fast - Old gen GC (Major GC): Infrequent, slower



2. Reference Types

Strong Reference (Default)

```
Object obj = new Object(); // Strong reference
```

Characteristics: - Never GC'd while reachable - Normal Java references - Only GC'd when no strong references exist

Weak Reference

```
WeakReference<LargeObject> weakRef = new WeakReference<>(obj);
obj = null; // Remove strong reference
System.gc();
weakRef.get(); // Returns null after GC
```

Characteristics: - GC'd in next GC cycle even if memory available - Doesn't prevent GC - `get()` may return null

Use Cases: - Caches that shouldn't prevent GC - Canonicalizing mappings (WeakHashMap)

Soft Reference

```
SoftReference<LargeObject> softRef = new SoftReference<>(obj);
```

Characteristics: - GC'd only when memory is low - JVM tries to keep them as long as possible - `get()` may return null after GC

Use Cases: - Memory-sensitive caches - Image caches - Any cache that can be recomputed

Phantom Reference

```
ReferenceQueue<Object> queue = new ReferenceQueue<>();  
PhantomReference<Object> phantomRef = new PhantomReference<>(obj, queue);
```

Characteristics: - `get()` always returns null - Enqueued AFTER object is finalized - Used for cleanup actions

Use Cases: - Post-mortem cleanup - Tracking object collection - Alternative to `finalize()`

3. Memory Leaks in Java

Yes, Java Can Have Memory Leaks!

Definition: Objects that should be GC'd but aren't because they're still referenced

Common Causes

1. Static Collections

```
// PROBLEM  
public class Cache {  
    private static List<Object> cache = new ArrayList<>();  
  
    public static void add(Object obj) {  
        cache.add(obj); // Never removed!  
    }  
}
```

Solution: Clear when done, use `WeakHashMap`, or bounded cache

2. Unclosed Resources

```
// PROBLEM  
Connection conn = dataSource.getConnection();  
// forgot to close!  
  
// SOLUTION  
try (Connection conn = dataSource.getConnection()) {  
    // Auto-closed  
}
```

3. Listeners and Callbacks

```
// PROBLEM
eventSource.addListener(this);
// Never removed, keeps 'this' alive

// SOLUTION
eventSource.removeListener(this); // In cleanup method
```

4. ThreadLocal

```
// PROBLEM
private static ThreadLocal<BigObject> threadLocal = new ThreadLocal<>();
// In thread pool, threads reused, BigObject never released

// SOLUTION
threadLocal.remove(); // Always remove when done
```

5. Inner Classes

```
// PROBLEM
public class Outer {
    private byte[] data = new byte[1024*1024];

    class Inner { // Holds reference to Outer!
    }
}

// SOLUTION
static class Inner { // No implicit reference
}
```

Detection

Tools: - VisualVM (heap dumps) - JProfiler - Eclipse MAT (Memory Analyzer Tool) - YourKit

Symptoms: - Heap usage grows over time - OutOfMemoryError - Frequent full GCs with no memory freed

4. Garbage Collectors

Types of GC Events

1. **Minor GC:** Cleans young generation (fast, frequent)
2. **Major GC:** Cleans old generation (slower)
3. **Full GC:** Cleans entire heap (slowest, causes "stop-the-world" pause)

GC Algorithms

1. Serial GC (`-XX:+UseSerialGC`)

Characteristics: - Single-threaded - "Stop-the-world" for both young and old gen - Simple, low overhead

When to use: Small apps, single-CPU systems, client apps

Pros: Low memory footprint, simple **Cons:** Long pauses

2. Parallel GC (`-XX:+UseParallelGC`)

Characteristics: - Multi-threaded GC - "Stop-the-world" but uses multiple threads - Throughput-focused

When to use: Batch processing, number crunching, throughput > latency

Pros: High throughput, good for multi-core **Cons:** Longer pauses than low-latency GCs

Tuning:

```
-XX:ParallelGCThreads=4 # Number of GC threads
```

3. G1 GC (`-XX:+UseG1GC`) ★ **Default in Java 9+**

Characteristics: - Divides heap into regions - Prioritizes regions with most garbage - Predictable pause times - Concurrent marking

When to use: General-purpose, default choice, large heaps (>4GB)

Pros: - Balanced (throughput + latency) - Predictable pauses - Handles large heaps well

Cons: More CPU overhead than Parallel GC

Tuning:

```
-XX:MaxGCPauseMillis=200 # Target max pause time  
-XX:G1HeapRegionSize=16m # Region size
```

4. ZGC (`-XX:+UseZGC`) - Java 15+

Characteristics: - Ultra-low latency (<10ms pauses) - Concurrent (most work done concurrently) - Scales to multi-TB heaps - Colored pointers

When to use: Low-latency requirements, large heaps, real-time systems

Pros: - Pause times independent of heap size - Very low latency

Cons: - Higher memory usage - More CPU overhead - Requires Java 15+

5. Shenandoah GC (`-XX:+UseShenandoahGC`)

Characteristics: - Similar to ZGC in goals - Concurrent compaction - Low pause times

When to use: Alternative to ZGC for low latency

Choosing a GC

Requirement	Recommended GC
Small app, single CPU	Serial GC
Batch processing, throughput	Parallel GC
General purpose	G1 GC
Low latency (<10ms)	ZGC or Shenandoah
Default (don't know)	G1 GC

5. GC Tuning

Heap Sizing

```
# Initial and max heap
-Xms2g -Xmx2g

# Young generation size
-Xmn512m

# Why set min=max?
# Avoids heap resizing, which causes full GC
```

GC Logging (Java 9+)

```
# Basic logging
-Xlog:gc

# Detailed logging
-Xlog:gc*

# Log to file
-Xlog:gc*:file=gc.log

# With timestamps
-Xlog:gc*:file=gc.log:time,uptime,level,tags
```

Common Tuning Flags

```
# Heap dump on OOM
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/path/to/dumps

# Print GC details (Java 8)
-XX:+PrintGCDetails -XX:+PrintGCTimeStamps

# Exit on OOM
-XX:+ExitOnOutOfMemoryError

# G1 specific
-XX:MaxGCPauseMillis=200
-XX:G1ReservePercent=10

# ZGC specific
-XX:ZAllocationSpikeTolerance=5
```

Tuning Methodology

1. **Measure First:** Run with GC logging
2. **Identify Issues:** High pause times? Frequent GC? OOM?
3. **Adjust One Thing:** Change one parameter at a time
4. **Measure Again:** Compare results
5. **Iterate:** Repeat until goals met

Red Flags

⚠️ **Full GC every few seconds** → Memory leak or heap too small ⚠️ **Long pause times** → Wrong GC or heap too large ⚠️ **OutOfMemoryError** → Leak, need more memory, or wrong sizing

6. Monitoring & Tools

Command-Line Tools

```
# Monitor GC in real-time
jstat -gc <pid> 1000 # Every 1 second

# Heap histogram
jmap -histo <pid>

# Heap dump
jmap -dump:format=b,file=heap.bin <pid>

# Thread dump
jstack <pid>
```

GUI Tools

- **VisualVM:** Free, bundled with JDK
- **JConsole:** Free, bundled with JDK
- **Eclipse MAT:** Heap dump analysis
- **JProfiler:** Commercial, powerful
- **YourKit:** Commercial

Production Monitoring

Metrics to track: - Heap usage (used, committed, max) - GC frequency (minor, major, full) - GC pause times (avg, max, p99) - Object allocation rate - Old gen growth rate

Tools: - Prometheus + Grafana - Datadog, New Relic, AppDynamics - Micrometer (metrics library)

Best Practices

Memory Management

1.  Use **try-with-resources** for `AutoCloseable`
2.  Avoid **`finalize()`** - deprecated, unreliable
3.  **Clear collections** when done
4.  **Remove listeners** in cleanup
5.  Use **weak/soft refs** for caches
6.  **Monitor memory** in production

7. Profile before optimizing

GC Tuning

1.  **Start with defaults** (G1 GC usually fine)
 2.  **Set -Xms = -Xmx** to avoid resizing
 3.  **Enable GC logging** always in production
 4.  **Measure, don't guess**
 5.  **One change at a time**
 6.  **Don't call `System.gc()`** - let JVM decide
 7.  **Don't over-tune** - premature optimization
-

Summary

Concurrency Key Points

- Use **ExecutorService**, not manual threads
- **CompletableFuture** for modern async
- **Atomic** for lock-free counters
- **synchronized** or **ReentrantLock** for critical sections
- Watch for **deadlocks** (lock ordering)
- Use **coordination utilities** (CountDownLatch, etc.)

GC Key Points

- Objects live on **heap**, managed by **GC**
 - **Strong refs** prevent GC, **weak refs** don't
 - Memory leaks occur when holding **unwanted references**
 - **G1 GC** is good default
 - **ZGC** for ultra-low latency
 - Always **monitor GC** in production
 - **Tune only when needed**
-
-

Chapter 6: Atomic Variables and CAS

6.1 Compare-And-Swap (CAS) Operation

The Foundation of Lock-Free Programming

CAS Operation is an atomic CPU instruction:

```
boolean compareAndSwap(int expected, int newValue) {  
    // ALL AS ONE ATOMIC OPERATION:  
    if (currentValue == expected) {  
        currentValue = newValue;  
        return true;  
    } else {  
        return false;  
    }  
}
```

How it works: 1. Read current value 2. Compare with expected value 3. If match, update to new value 4. Return whether update succeeded

All in ONE atomic CPU instruction - no locks needed!

CAS in Action

```
AtomicInteger counter = new AtomicInteger(0);  
  
// Thread-safe increment using CAS  
public void increment() {  
    int oldValue, newValue;  
    do {  
        oldValue = counter.get();  
        newValue = oldValue + 1;  
    } while (!counter.compareAndSet(oldValue, newValue));  
    // Retry if another thread changed value  
}
```

Why no race condition? - If another thread changed value, CAS fails - Loop retries with new value - Eventually succeeds

6.2 Atomic Classes Family

java.util.concurrent.atomic Package

Atomic Numeric: - `AtomicInteger` - `AtomicLong` - `AtomicBoolean`

Atomic References: - `AtomicReference<T>` - `AtomicStampedReference<T>` (solves ABA problem) - `AtomicMarkableReference<T>`

Atomic Arrays: - `AtomicIntegerArray` - `AtomicLongArray` - `AtomicReferenceArray<T>`

Atomic Field Updaters: - `AtomicIntegerFieldUpdater` - `AtomicLongFieldUpdater` - `AtomicReferenceFieldUpdater`

AtomicInteger Deep Dive

Code Example (from `AtomicVariablesDemo.java`):

```
AtomicInteger atomicCounter = new AtomicInteger(0);

// Create 10 threads, each incrementing 1000 times
Thread[] threads = new Thread[10];
for (int i = 0; i < 10; i++) {
    threads[i] = new Thread(() -> {
        for (int j = 0; j < 1000; j++) {
            atomicCounter.incrementAndGet(); // Thread-safe, lock-free!
        }
    });
}

for (Thread thread : threads) {
    thread.start();
}

for (Thread thread : threads) {
    thread.join();
}

System.out.println("Expected: 10,000");
System.out.println("Actual: " + atomicCounter.get());
// Output: Actual: 10,000 ✓ CORRECT!
```

Common Methods:

```

AtomicInteger value = new AtomicInteger(10);

// Get and Set
int current = value.get();           // 10
value.set(20);                       // Set to 20

// Increment/Decrement
int result = value.incrementAndGet(); // ++value, returns 21
result = value.getAndIncrement();     // value++, returns 21, becomes 22
result = value.decrementAndGet();     // --value, returns 21
result = value.getAndDecrement();     // value--, returns 21, becomes 20

// Add
result = value.addAndGet(5);          // value += 5, returns 25
result = value.getAndAdd(5);          // Returns 25, becomes 30

// Compare and Set (CAS)
boolean updated = value.compareAndSet(30, 100);
System.out.println(updated);          // true
System.out.println(value.get());      // 100

// Try again with wrong expected value
updated = value.compareAndSet(50, 200);
System.out.println(updated);          // false (expected 50, but actual is 100)
System.out.println(value.get());      // Still 100

```

AtomicReference Example

```
class UserStatus {
    final String username;
    final String status;

    UserStatus(String username, String status) {
        this.username = username;
        this.status = status;
    }
}

AtomicReference<UserStatus> statusRef = new AtomicReference<>(
    new UserStatus("John", "OFFLINE")
);

// Thread 1: Login
Thread loginThread = new Thread(() -> {
    UserStatus current = statusRef.get();
    UserStatus updated = new UserStatus(current.username, "ONLINE");
    if (statusRef.compareAndSet(current, updated)) {
        System.out.println("Login successful: Updated to ONLINE");
    }
});

// Thread 2: Logout (with slight delay)
Thread logoutThread = new Thread(() -> {
    sleep(50);
    UserStatus current = statusRef.get();
    UserStatus updated = new UserStatus(current.username, "OFFLINE");
    if (statusRef.compareAndSet(current, updated)) {
        System.out.println("Logout successful: Updated to OFFLINE");
    }
});

loginThread.start();
logoutThread.start();

loginThread.join();
logoutThread.join();

System.out.println("Final status: " + statusRef.get().status);
```

Output:

```
Login successful: Updated to ONLINE
Logout successful: Updated to OFFLINE
Final status: OFFLINE
```

6.3 ABA Problem and Solutions

What is the ABA Problem?

Scenario: 1. Thread 1 reads value A 2. Thread 2 changes A → B → A (back to A!) 3. Thread 1's CAS succeeds (sees A, doesn't know it changed!)

Example:

```
Stack Top → Node A → Node B → Node C

Thread 1: Reads A, wants to pop (make top = B)
Thread 2: Pops A, pops B, pushes A back
          Stack Top → Node A → Node C

Thread 1: CAS(expected=A, new=B) succeeds! ← WRONG!
          Now B.next is undefined!
```

Solution: AtomicStampedReference

Adds a version stamp to detect changes:

```
AtomicStampedReference<UserStatus> statusRef =
    new AtomicStampedReference<>(
        new UserStatus("John", "OFFLINE"),
        0 // Initial stamp
    );

// Get value and stamp
int[] stampHolder = new int[1];
UserStatus current = statusRef.get(stampHolder);
int stamp = stampHolder[0];

// CAS with stamp check
UserStatus updated = new UserStatus(current.username, "ONLINE");
boolean success = statusRef.compareAndSet(
    current,          // expected reference
    updated,          // new reference
    stamp,            // expected stamp
    stamp + 1         // new stamp
);

// If reference changed back to same value but stamp different, CAS fails!
```

Reference: See

```
src/main/java/org/example/javaconcurrency/concurrency/atomic/AtomicVariablesDemo.java
```

6.4 Performance Comparison

Atomic vs Synchronized

From AtomicVariablesDemo.java:

```
// Test: 10 threads, each incrementing 100,000 times

// With AtomicInteger:
AtomicInteger atomicCounter = new AtomicInteger(0);
// Time: 45ms
// ✅ Lock-free, very fast

// With synchronized:
class SynchronizedCounter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
}
// Time: 85ms
// ⚠️ Slower due to lock overhead

// Speedup: ~89% faster with Atomic!
```

When Atomic is Faster: - ✅ Low contention (few threads competing) - ✅ Short critical sections -
✅ Simple operations (increment, compareAndSet)

When Synchronized is Better: - Complex operations needing multiple steps - High contention scenarios - Need visibility guarantees for multiple variables

Performance Characteristics

Operation	synchronized	AtomicInteger
Simple increment	100ns	15ns
High contention	Blocking	Spinning (may be slower)
Composability	Easy	Harder
Deadlock risk	Yes	No

Best Practices: - ✅ Use Atomic for **counters, flags, references** - ✅ Use synchronized for **complex multi-step operations** - ✅ Use locks when you need to **coordinate multiple variables**

Chapter 7: Coordination Utilities (Brief Overview)

7.1 CountdownLatch

Purpose: Wait for N operations to complete.

```
CountDownLatch latch = new CountDownLatch(3); // 3 services

// Worker threads
new Thread(() -> { startDatabase(); latch.countDown(); }).start();
new Thread(() -> { startCache(); latch.countDown(); }).start();
new Thread(() -> { startAPI(); latch.countDown(); }).start();

// Main thread waits
latch.await(); // Blocks until count reaches 0

System.out.println("All services started!");
```

Reference:

```
src/main/java/org/example/javaconcurrency/concurrency/coordination/CountDownLatchDemo.java
```

7.2 CyclicBarrier

Purpose: N threads wait for each other, reusable.

```
CyclicBarrier barrier = new CyclicBarrier(4, () -> {
    System.out.println("All players ready! Starting game...");
});

// 4 player threads each call:
barrier.await(); // Wait for all 4

// Can be reused for multiple rounds
```

Reference:

```
src/main/java/org/example/javaconcurrency/concurrency/coordination/CyclicBarrierDemo.java
```

7.3 Semaphore

Purpose: Limit concurrent access (resource pool).

```
Semaphore connectionPool = new Semaphore(5); // 5 permits

connectionPool.acquire(); // Take permit
try {
    // Use database connection
} finally {
    connectionPool.release(); // Return permit
}
```

Reference:

```
src/main/java/org/example/javaconcurrency/concurrency/coordination/SemaphoreDemo.java
```

7.4 BlockingQueue

Purpose: Producer-consumer pattern.

```
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(10);

// Producer
queue.put(task); // Blocks if full

// Consumer
Task task = queue.take(); // Blocks if empty
```

Reference:

```
src/main/java/org/example/javaconcurrency/concurrency/coordination/BlockingQueueDemo.java
```

Chapter 8: Common Pitfalls

8.1 Deadlock

From DeadlockDemo.java:

```
// PROBLEM: Deadlock
Object lock1 = new Object();
Object lock2 = new Object();

Thread t1 = new Thread(() -> {
    synchronized(lock1) {
        sleep(100);
        synchronized(lock2) { } // Waits for lock2
    }
});

Thread t2 = new Thread(() -> {
    synchronized(lock2) {
        sleep(100);
        synchronized(lock1) { } // Waits for lock1
    }
});

t1.start();
t2.start();
// DEADLOCK! Both threads wait forever
```

SOLUTION: Lock Ordering:

```
// Always acquire locks in same order
Account first = account1.getId().compareTo(account2.getId()) < 0
    ? account1 : account2;
Account second = first == account1 ? account2 : account1;

synchronized(first) {
    synchronized(second) {
        // Transfer money
    }
}
```

Reference: `src/main/java/org/example/javaconcurrency/concurrency/pitfalls/DeadlockDemo.java`

5. Atomic Variables

Lock-Free Thread Safety

Problem with locks: Overhead, contention, deadlock risk

Solution: Atomic variables using CAS (Compare-And-Swap)

```
AtomicInteger counter = new AtomicInteger(0);

counter.incrementAndGet(); // Atomic increment
counter.compareAndSet(10, 20); // Set to 20 if currently 10
```

Types: - `AtomicInteger` , `AtomicLong` - Numeric atomics - `AtomicBoolean` - Boolean flag - `AtomicReference<T>` - Object reference - `AtomicIntegerArray` - Array elements

When to use: - Counters and accumulators - Flags and state variables - Simple lock-free algorithms - Better performance than synchronized for simple ops

6. Coordination Utilities

CountDownLatch

Purpose: One/more threads wait for N operations to complete

```
CountDownLatch latch = new CountDownLatch(3);

// Workers
new Thread() -> { doWork(); latch.countDown(); }.start();
new Thread() -> { doWork(); latch.countDown(); }.start();
new Thread() -> { doWork(); latch.countDown(); }.start();

// Main waits
latch.await(); // Blocks until count reaches 0
```

Use Cases: Service startup, parallel initialization

CyclicBarrier

Purpose: N threads wait for each other at barrier point

```
CyclicBarrier barrier = new CyclicBarrier(3, () -> {
    System.out.println("All arrived!");
});

// Each thread calls:
barrier.await(); // Waits until all 3 arrive
```

Difference from CountDownLatch: Reusable, threads wait for each other

Use Cases: Iterative algorithms, multi-phase computations

Semaphore

Purpose: Limit concurrent access to resource

```
Semaphore pool = new Semaphore(5); // 5 permits

pool.acquire(); // Take permit (blocks if none available)
try {
    // Use resource
} finally {
    pool.release(); // Return permit
}
```

Use Cases: Connection pools, rate limiting

BlockingQueue

Purpose: Thread-safe queue with blocking operations

```
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(10);

// Producer
queue.put(task); // Blocks if full

// Consumer
Task task = queue.take(); // Blocks if empty
```

Types: - `ArrayBlockingQueue` - Bounded, array-based - `LinkedBlockingQueue` - Optionally bounded, linked nodes - `PriorityBlockingQueue` - Ordered by priority - `SynchronousQueue` - No capacity, hand-off

Use Cases: Producer-consumer, task queues, message passing

7. Common Pitfalls

Deadlock

Definition: Two or more threads wait for each other indefinitely

Conditions (all must be true): 1. Mutual exclusion 2. Hold and wait 3. No preemption 4. Circular wait

Example:

```
// Thread 1
synchronized(lock1) {
    synchronized(lock2) { } // Waits for lock2
}

// Thread 2
synchronized(lock2) {
    synchronized(lock1) { } // Waits for lock1
}

// DEADLOCK!
```

Prevention: 1. **Lock ordering:** Always acquire locks in same order 2. **Timeouts:** Use tryLock(timeout) 3. **Deadlock detection:** Monitor thread dumps

Visibility Issues

Problem: Thread doesn't see updates from other threads

Solution: Use `volatile`

```
private volatile boolean flag = false;
```

volatile guarantees: - Visibility (all threads see latest value) - No caching in CPU registers - No reordering around volatile reads/writes

When to use: Flags, state variables (not for compound operations)